Ghulam Ishaq Khan Institute of Engineering Sciences and Technology

Faculty of Computer Science and Engineering

DineHub

Your Culinary Marketplace

Supervisor: Mr. Muhammad Sajid Ali

Co-Supervisor: Mr. Qasim Riaz

Group Members:

2021015 Mr. Abdul Moiz Ghumman

2021486 Mr. Muhammad Waiz Hamid

2021535 Mr. Rayyan Hassan

Certificate of Approval

It is certified that the work presented in this report was performed by **Abdul Moiz Ghumman**, **Muhummad Waiz Hamid**, and **Rayyan Hassan** under the supervision of **Muhammad Sajid Ali, and Muhammad Qasim Riaz.** The work is adequate and lies within the scope of the BS degree in Computer Engineering at Ghulam Ishaq Khan Institute of Engineering Sciences and Technology.

Muhammad Sajid Ali

(Advisor)

Muhammad Qasim Riaz

(Co-Advisor)

Prof. Dr. Qadeer Ul Hassan

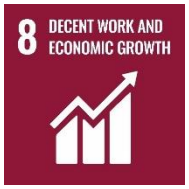
(Dean)

SDGs Inclusion

SDG 2: Zero Hunger

DineHub combats hunger on the GIKI campus by expediting meal ordering, ensuring on-time deliveries, and eliminating food waste through effective inventory and menu management for vendors.

SDG 8: Decent Work and Economic Growth



The platform promotes entrepreneurship among students, vendors, and passengers by providing digital tools and a regulated ecosystem that supports campus-wide economic activity.

SDG 9: Industry, Innovation, and Infrastructure

DineHub promotes innovation, creating a strong digital infrastructure that simplifies the meal delivery process by including a friendly user system.

SDG 11: Sustainable Cities and Communities

DineHub improves campus life by streamlining order processes and encouraging a smart, connected community.

---	---
Muhammad Sajid Ali	**Muhammad Qasim Riaz**
(Advisor)	(Co-Advisor)

FINAL YEAR PROJECT MAPPING TO

COMPLEX ENGINEERING/COMPUTING PROBLEM ATTRIBUTES

It is to certify here that the final year design project (FYDP) entitled DineHub is categorized as a complex engineering/computing problem (CEP) based on the preamble (in-depth engineering knowledge) and involvement of the following attributes:

Attribute	**Justification**
Range of Conflicting Requirements	Minimizing delivery time while reducing operational cost, along with providing real-time order status updates, presents multiple conflicting requirements.
Depth of Analysis Required	Required deep analysis of order routing, database optimization for real-time operations.
Depth of Knowledge Required	Knowledge in Flutter app development, Node.js backend, MongoDB, REST APIs, Langchain, Pinecone Vectors, Nginx and SSL.
Familiarity of Issues	Custom solutions were needed for GIKI's unique environment including offline delivery tracking
Extent of Applicable Codes	System followed food safety and delivery guidelines and the best data protection practices
Extent of Stakeholder Involvement and Level of Conflicting Requirements	Students, sellers, riders, and admin were all considred ensuring a smooth app experience

Consequences	Failure in delivery could lead to financial loss, unfair billing, or loss of user trust.
Interdependence	The app's functionality relied on close coordination between mobile apps (user, seller), backend services and database consistency.

Muhammad Sajid Ali

(Advisor)

Muhammad Qasim Riaz

(Co-Advisor)

ABSTRACT

DineHub is a centralized food e-marketplace specifically designed for the Ghulam Ishaq Khan Institute (GIKI) campus. The application addresses the inefficiencies of the current manual ordering systems by digitizing the entire process. There are separate interfaces for users and vendors. DineHub ensures seamless coordination and transparency across all parties.

The system uses modern technologies like Node.js and MongoDB to create a scalable and secure service. A complete end-to-end ordering flow was constructed: users can browse the menu, place an order, pay, and afterwards they can leave a review which greatly helps in refining customer satisfaction. It accepts a variety of payment methods, including Cash on Delivery, MasterCard, Use of banking apps, SadaPay, Easypaisa, and JazzCash, which reduces dependency on cash transactions. Furthermore, we implemented a digital wallet feature that allows payment even when offline, enhancing ease of payment devoid of internet connection.

Artificial Intelligence has now been embedded into the app, bringing a great advancement toward modernization. An intelligent chatbot has been incorporated that addresses all prompts GIKI it helps users make decisions like what they should eat and even assists them with placing their orders.

In order to enhance the flexibility and usability of the app, we improved features designed around menu controls menu items can now be added or removed with greater precision. Collectively, these factors contribute toward an advanced, streamlined, intuitive, and smart food ordering experience.

To ensure better organization and scalability of the system, Docker containers were created for each essential part of the application. These included Nginx as a reverse proxy to manage and route traffic, Node.js for the backend, and MongoDB for the database. Application is hosted live on the GIKI server and is now reachable through the domain **dinehub.giki.edu.pk**. The application is accessible through any browser by visiting the domain. For added ease of use, we developed and published an APK version of the app which allows mobile users to access its features with a single click.

This architecture design guarantees hassle-free monitoring and access to the app at any time. Due to its Progressive Web App (PWA) architecture, DineHub can be easily accessed on iOS and Android smartphones without requiring users to download it from an app store. It provides a quick, app-like experience straight from the browser, is completely responsive, and functions on all screen sizes. It guarantees safe data transfer and user privacy with integrated SSL encryption and HTTPS. This method combines the flexibility and reach of the web with the ease of use of a mobile app.

ACKNOWLEDGEMENTS

We would first like to start off by thanking the one and only Allah Almighty for giving us the perseverance, knowledge, and determination to go through and complete this project despite the various hurdles.

We would like to express our heartfelt gratitude to the following individuals and institutions for their invaluable support throughout the course of this project.

Primarily, we are thankful to our advisor, **Muhammad Sajid Ali**, for his continuous guidance and support. His expertise and insightful feedback were crucial to the success of this project. We would also like to extend our gratitude to our co-advisor, **Qasim Riaz**, for his contributions to the project's direction.

We also like to thank Ghulam Ishaq Khan Institute of Engineering Sciences and Technology for providing the required resources, such as access to the server and domain where the app is installed and numerous tools that were vital to the project's success.

We would like to thank our fellow friends who helped us during the requirements gathering and testing phase of the project. This helped us streamline the project for the final release.

Finally, we are grateful to our families for their constant support and encouragement, which kept us motivated throughout this process.

TABLE OF CONTENTS

Contents

Chapter I: Introduction 13
1.1 Purpose and Motivation 13
1.2 Problem Statement 13
1.3 Product Scope 14
1.4 Objectives 15
Chapter II: Literature Review and Survey 16
2.1 Literature Survey 16
2.2 Existing Systems 16
2.3 Comparison Matrix 17
2.4 Requirement Gathering 18
2.4.1 Student Feedback 18
2.4.2 Vendor Feedback 20
2.4.3 Faculty Feedback 20
2.4.4 Rider Feedback 20
2.5 Personas 21
2.5.1 User Personas 22
2.5.2 Vendor Personas 23
2.5.3 Rider Personas 24
Chapter III: Design 25
3.1 Product Perspective 25
3.2 Product Functions 25
3.3 Product Characteristics 26
3.4 Constraints 26
3.5 Assumptions and Dependencies 28

3.6 Functional Requirements 28
3.7 Nonfunctional Requirements & Software 36
3.7.1 System Attributes 36
3.7.2 Performance Requirements 36
3.7.3 Security Requirements 37
Chapter IV: Project Design/Architecture 38
4.1 Use Case View 39
4.1.1 User Use Case View 39
4.1 Vendor Use Case View 40
4.2 Logical View 41
4.3 Process View 41
4.3.1 User Sequence Diagram 41
4.3.2 Vendor Sequence Diagram 43
4.4 Physical View 44
4.5 User Interface Design 46
4.5.1 User App 46
4.5.2 Seller App 53
4.6 Context Diagram 57
Chapter V: Proposed Solution 58
5.1 Development Methodology 58
5.2 System Implementation and Technologies 59
5.3 Containerized Deployment 60
5.4 GitHub Repository and Collaboration 61
5.5 API Documentation 63
5.6 Key Features and Security Measures 64
Chapter VI: Results and Discussions 64

6.1 System Implementation Outcomes 64
6.2 User Testing 66
6.3 User Feedback 67
6.3 Challenges Faced 67
Chapter VII: Conclusion and Future Work 69
7.1 Conclusion 69
7.2 Future Work 69
GLOSSARY 70
REFERENCES 71
APPENDIX 73

LIST OF FIGURES

Figure 1 Problem Statement Diagram 14
Figure 2 Comparison Matrix 17
Figure 3 Survey Q1 18
Figure 4 Survey Q2 19
Figure 5 Survey Q5 19
Figure 6 User Persona 22
Figure 7 Vendor Persona 23
Figure 8 Rider Persona 24
Figure 9 Architecture Diagram 38
Figure 10 User Use Case View 39
Figure 11 Vendor Use Case View 40
Figure 12 Class Diagram 41
Figure 13 User Sequence Diagram 42
Figure 14 Vendor Sequence Diagram 43
Figure 15 User Activity Diagram 44
Figure 16 Vendor Activity Diagram 45
Figure 17 Flash Screen 46
Figure 18 Login Page 47

Figure 19 Register Page 47
Figure 20 Menu Page 48
Figure 21 Items Page 48
Figure 22 Item Details 49
Figure 23 Order History 49
Figure 24 Address Page 50
Figure 25 My Cart 50
Figure 26 Order Place Screen 51
Figure 27 Chatbot Screen 52
Figure 28 App Bar 53
Figure 29 Menu Screen 53
Figure 30 Item Details 54
Figure 31 Uploading Items Screen 54
Figure 32 Payment Details Screen 55
Figure 33 New Orders Screen 55
Figure 34 Analytics Screen 56
Figure 35 Order Details Screen 56
Figure 36 Context Diagram 57
Figure 37 Jira Screenshot 58
Figure 38 Containers 61
Figure 39 Contributors 61
Figure 40 Commits Screenshot 62
Figure 41 Readme File 62
Figure 42 Postman 63

LIST OF TABLES

Table 1 Functional Requirements 29
Table 2 User Registration 30
Table 3 User Information Updating 31
Table 4 Vendor Registration 32
Table 5 Order placement 33

Table 6 Payment Process 34
Table 7 Vendor Order Acceptance 35
Table 8 Technology Stack 59

Chapter I: Introduction

1.1 Purpose and Motivation

This report describes the requirements and specifications of DineHub, an online food ordering and pickup portal specifically tailored for the GIKI campus.

Instead of searching for problems outside, we chose to fix our problem from home. We have developed an app that improves dining life for students and staff in GIKI. As student developers, we are keen on implementing innovative technology towards solving significant issues at our own university. Our project is fully deployed and functional in campus, making this an excellent final-year project.

1.2 Problem Statement

Students and staff at GIKI face persistent challenges in ordering food due to the absence of a centralized digital platform. Currently, users must call restaurants individually, which often leads to missed calls and lost orders, especially during peak hours or network issues. The reliance on cash payments further complicates the process, as ATMs frequently run out of cash, and vendors struggle with payment tracking and financial reconciliation due to paper-based systems. Riders face similar inefficiencies, manually recording orders and contacts, while societies and event organizers encounter difficulties with bulk orders, reimbursement, and the lack of digital receipts or order history. Overall, the current system is outdated, error-prone, and time-consuming, highlighting the urgent need for an integrated solution that streamlines ordering, payments, and communication for all stakeholders on campus.

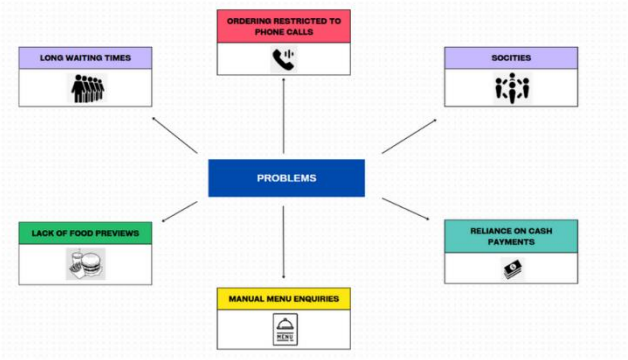


Figure 1 Problem Statement Diagram

1.3 Product Scope

The scope of DineHub focuses on delivering a centralized and efficient food e-marketplace tailored for the GIKI community, aiming to streamline the food ordering experience for students, faculty, and on-campus vendors. The platform provides two core interfaces: a User App and a Seller App. The User App enables customers to browse digital menus, place orders, track delivery status in real time, and provide feedback on their experience. The Seller App allows food vendors to manage orders, update menu availability, and track sales, all from a single, user-friendly interface. This dual-interface system ensures better communication, improved order accuracy, and reduced reliance on manual, error-prone processes.

DineHub also incorporates several advanced features to enhance usability and accessibility. These include an AI-powered chatbot for instant customer

support, support for multiple payment methods including digital wallets and offline cash payments, and a feedback system to promote service quality and accountability. The platform also facilitates smoother workflows for bulk or departmental orders by enabling digital coordination with societies and campus entities. While initially designed for deployment within GIKI, the system is architected to be scalable and adaptable for other similar environments, such as universities, residential campuses, or institutional food courts, with minimal customization.

1.4 Objectives

- Develop a centralized food delivery and pickup platform for the GIKI campus.
- Eliminate the need for phone calls or SMS to place orders.
- Prevent order mix-ups or forgotten orders with a digital system.
- Implement order status tracking and push notifications.
- Enable admins to allocate funds to faculty/societies for event catering.
- Support multiple payment methods using APIs.
- In case of network issues, be able to perform offline payments.
- Ensure scalability and reliability with a robust backend architecture.
- Implementation of Chatbot to improve user experience.
- Addition of feedback to improve and ensure customer satisfaction.
- Facilitate efficient order and delivery management for sellers and riders.
- Deployment of the app on GIKI server.
- Develop a Progressive Web App (PWA) compatible with both iOS and Android.
- Use of Docker Containers to ease the Deployment process.

- To eliminate security concerns, apply SSL and HTTPS protocols.
- Add Data Analytics, so vendor can maintain Inventory.
- Implementation of JSON Web tokens for authentication and to maintain stateless sessions.

Chapter II: Literature Review and Survey

The expansion of food delivery services has significantly transformed consumer behaviour in student-rich environments like university campuses. DineHub takes inspiration from leading platforms such as Uber Eats, Grubhub, and Foodpanda, integrating these best practices into its design. By utilizing a scalable backend architecture with Node.js and MongoDB, using reverse proxy and Docker containers to ensure robust performance and adaptability.

2.1 Literature Survey

The need for dependable and effective food delivery apps is rising because of the profound effects that technology is having on all industries, including the food industry. In recent years, there has been a growing literature focused on the development and implementation of such systems.

Studies have highlighted the importance of food delivery apps. Food delivery apps bring many benefits to the students and catering business.

2.2 Existing Systems

The food delivery market is dominated by established platforms such as Uber Eats, Grubhub, and Foodpanda, providing comprehensive solutions for urban areas. These platforms set a high barrier.

However, these existing platforms do not cater to the unique needs of university campuses, where affordability, convenience, and localized service are paramount. DineHub aims to fill this gap. We focus on the requirements of students, faculty, vendors, and admin, offering a tailored solution that prioritizes cost-effectiveness and seamless integration into campus life.

2.3 Comparison Matrix

Feature	Uber Eats	DoorDash	Grubhub	Zomato	Proposed System
Food Delivery	Yes	Yes	Yes	Yes	Yes
Pickup Option	No	Yes	Yes	Yes	Yes
Multiple Payment Methods	Yes	Yes	Yes	Yes	Yes
Real-Time Order Tracking	Yes	Yes	Yes	Yes	Yes
Event Expense Tracking	No	No	No	No	Yes
Initial Campus Focus	No	No	No	No	Yes (GIKI campus)
User-Friendly Interface	Yes	Yes	Yes	Yes	Yes
Future Market Expansion	Yes	Yes	Yes	Yes	Yes

Figure 2 Comparison Matrix

The above matrix suggests that we cannot use these existing platforms because the existing platforms are:

- **Closed Source**: These services are proprietary and are not open source, making it tough to adapt their systems.
- **Private Companies**: As private entities, they operate independently and would require their infrastructure and resources to establish a presence in our area, which is not feasible for our project.
- **Lack of Local Adaptation**: These platforms may not cater to the unique requirements of our campus environment, making it essential

to develop a tailored solution that addresses the specific needs of GIKI students and faculty.

2.4 Requirement Gathering

We gathered data from various stakeholders, including students, vendors, and faculty, to understand their experiences and challenges with the current food ordering system. Surveys and interviews were conducted among key stakeholders, which included students, vendors, riders, and faculty, in an attempt to gather inputs and understand their needs.

2.4.1 Student Feedback

Since students represent the largest population, we conducted a survey distributed across diverse groups. The survey consisted of seven questions, and we received valuable insights regarding their frustrations and needs. Here are the takeaways:

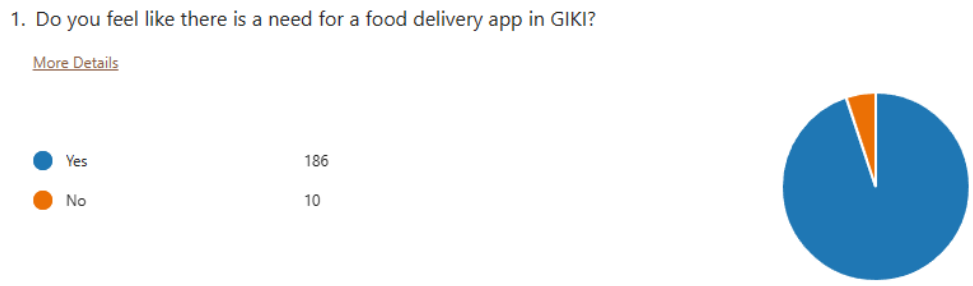


Figure 3 Survey Q1

2. How often do you order food via delivery?

More Details Insights

Almost daily	111
2-3 times a week	54
Once a week	21
Rarely	14

Figure 4 Survey Q2

5. Would you like to be able to order groceries from within the app?

More Details Insights

Yes	187
No	14

Figure 5 Survey Q5

This gave us statistics to look at. We asked a question to help us identify the most common problems faced by students while ordering food. The most common responses are below:

- "I sometimes have no service on my phone, so it becomes impossible for me to order food from my phone, and I have to ask other people, which is inconvenient."
- "Ordering food via calls and messages need a proper app."
- "Mostly I have to ask daily what items available and what price are while calling tuc. So, it would be great if I can see menu and prices on app while ordering making it much good experience."
- "It's hard to contact these restaurants at Giki through calls and sms."

- “Having to call lala every other second, sometimes phone is engaged sometimes other problem.”

2.4.2 Vendor Feedback

We also engaged with vendors to understand their challenges. Here are the key problems they reported:

- Vendors struggle with managing incoming orders manually, leading to confusion and errors.
- Reliance on cash payments can lead to delays.
- Difficulty in communicating with customers and riders, leading to misunderstandings about order specifics.
- Difficulty in managing and responding to customer reviews and feedback.

2.4.3 Faculty Feedback

Faculty members also faced similar challenges when ordering food and organizing events. Here are the problems they identified:

- Faculty members experienced difficulties in placing orders, often having to rely on phone calls that went unanswered.
- Organizing food for events was cumbersome, with issues in coordinating orders and payments.

2.4.4 Rider Feedback

Riders also shared their frustrations regarding the current system. Here are the key issues they reported:

- Riders often rely on paper for order details, leading to misplaced or unclear information during deliveries.

- Difficulty in communicating with vendors and customers can lead to misunderstandings about order specifics or delivery times.

This section outlines the user and system requirements needed to implement DineHub effectively, ensuring it meets the functional and non-functional expectations for users on the GIKI campus.

2.5 Personas

In order to gain a deeper insight into the various needs and issues of our stakeholders, we designed personas which reflect major user groups of the DineHub ecosystem. These characters are fictional but grounded in actual data and research insights we have collected. They assist us in becoming more empathetic with real users, seeing their issues, and designing solutions for them according to their individual requirements.

2.5.1 User Personas

The user persona is a student whose needs, frustrations, and expectations are showcased through their requirements such as swift ordering, guaranteed service, and convenient access to menus in a food delivery application.

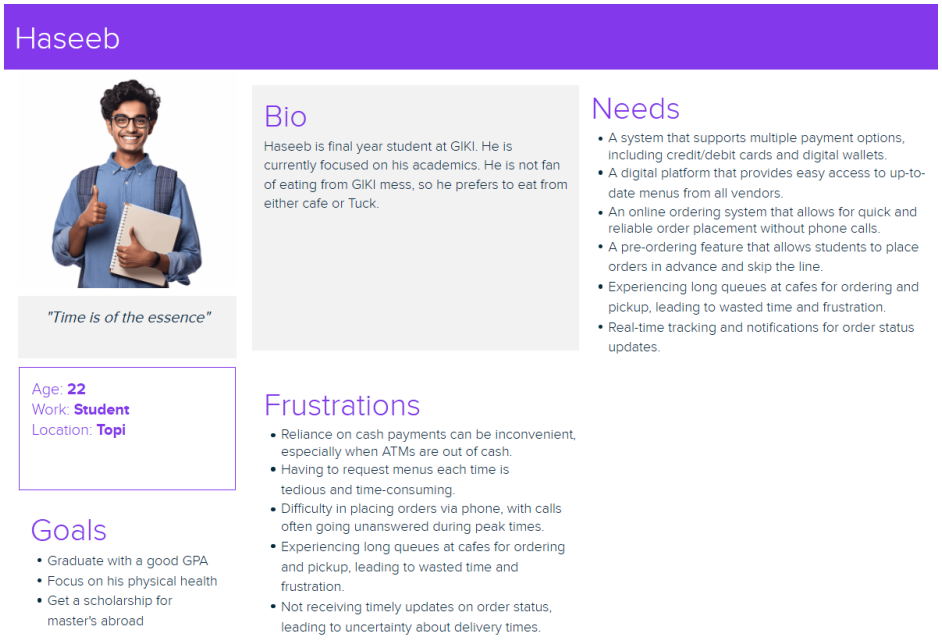


Figure 6 User Persona

2.5.2 Vendor Personas

The vendor persona is a student operating a food stand, illustrating their frustrations and requirements like keeping orders organized, inventory tracking, and getting payments in a timely manner through a convenient food delivery app.

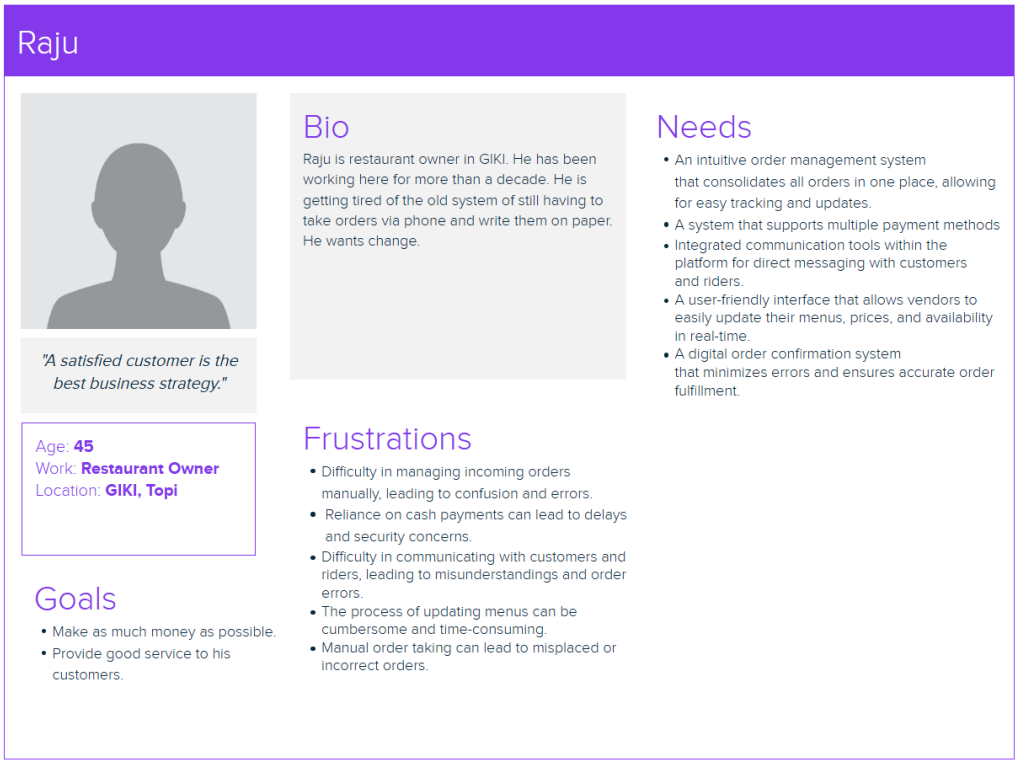


Figure 7 Vendor Persona

2.5.3 Rider Personas

The rider user persona outlines a person who is working as a delivery rider, thus highlighting their requirements and issues, such as the requirement for correct order details, best routes for delivery, timely payment, and an effective delivery management system.

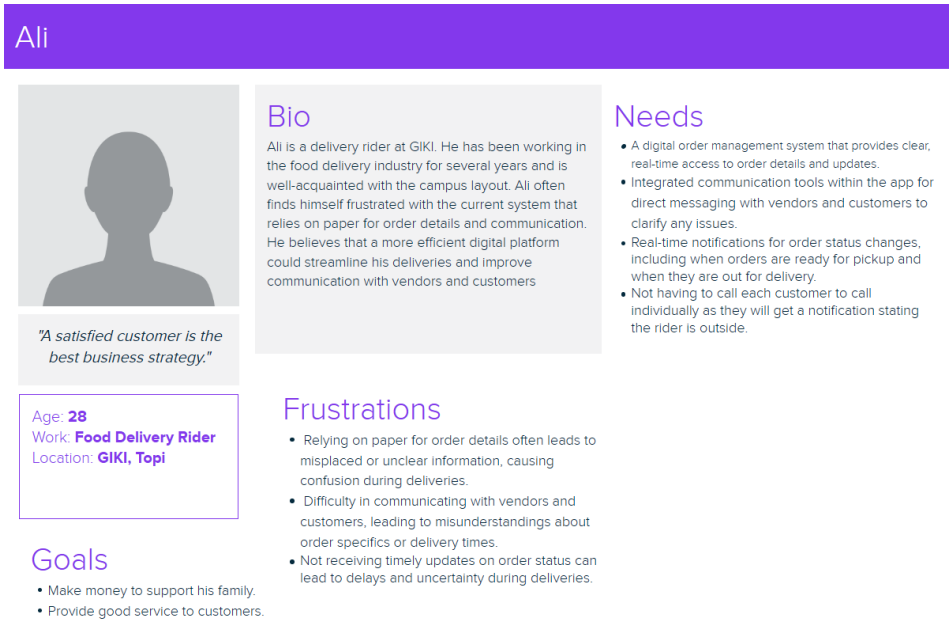


Figure 8 Rider Persona

Chapter III: Design

DineHub is an e-marketplace developed on Node.js tailored for the GIKI campus. It has a user app for students and faculty members and a seller app for vendors. The site offers order updates, push notifications, and multiple payment methods. DineHub will reduce queues at campus cafes after enabling pre-orders and electronic payments. It does not use phone calls or SMS to order, eliminating confusion and lost orders. The admin portal also enables the distribution of funds to societies and faculty for event catering. With a robust backend, DineHub offers scalability and reliability, using state-of-the-art technologies to address campus-relevant issues and enhance the learning experience for its developers.

3.1 Product Perspective

DineHub is designed specifically for campus use, with a major emphasis on catering to the special needs of GIKI campus dwellers. The platform provides localized delivery options that solve an existing problem for the students and staff. Through its customized design, DineHub hopes to improve the overall dining experience, minimize wait times, and simplify the food ordering process within the campus community.

3.2 Product Functions

- Enabling users to browse menus and place orders.
- Enabling users to provide feedback for future improvements.
- Supporting multiple payment methods, such as SadaPay, Easypaisa, and JazzCash.
- Supporting offline payments through wallet service.
- Allowing vendors to manage menus, orders, and pricing.

- Facilitating efficient delivery management for riders.
- Offering an admin portal for user and order management.
- Maintaining system logs for operational transparency.
- Managing user accounts, including registration and privilege updates.
- Providing a chatbot for convenience.

3.3 Product Characteristics

1. **Users**
    - Can browse menus, place orders, and track deliveries.
    - Can view order history and manage account settings.
    - Receives notifications for order status updates.
    - Provide feedback in form of ratings and text response.
    - Can use multiple payment methods for payment.
    - Use AI Chatbot for better interaction with the App.
2. **Vendor**
    - Can manage orders.
    - Update menus.
    - Set pricing.
    - Track and manage payments.
3. **System Administrator**
    - Can manage user accounts.
    - Verify vendor registrations.
    - Can oversee order processing.
    - Handle disputes.

3.4 Constraints

- **Mobile Platform Limitations**:

Due to the excessive costs associated with launching on the Play Store and App Store, DineHub will initially be offered as a web-based application. The App is deployed on Giki local server and is only accessible when being on Campus network. For that reason, stores like Play Store would need to test it and sign up to see features, functionalities and how useful this app would be. But they can't do that unless they have access to the network of GIKI. This approach may limit native mobile functionalities and user experience optimizations typically available in native apps. The app will be on a landing page where users may download the APK file or use the web application through the domain **dinehub. giki. edu. pk**. The authentication is quite secure as well, we included SSL certificates and ensured all connectivity occurs through HTTPS.

- **Network Speed and Reliability:**

  The application's performance is dependent on network speed and reliability. Slow or unstable internet connections can lead to delays in loading times and interruptions during the ordering and payment processes. The app can struggle because of low bandwidth, delayed latency and low throughput; these problems are made even worse on a busy network due to caused delays or more failed requests.

- **Integration with Third-Party APIs:**

Integrating with third-party APIs, such as payment gateways, chatbots, and messaging platforms, can be complex and time-consuming.

- **Campus-Specific Limitations:**

The service is initially for the GIKI campus, which may restrict scalability and user base expansion. Any expansion beyond the campus will require additional resources and adjustments to the platform.

- **Resource Constraints:**

As a student-led project, DineHub may face limitations in terms of available resources, including time, budget, and experience, which could impact the scope and pace of development.

3.5 Assumptions and Dependencies

- The system depends on reliable internet access across the campus to ensure seamless operation and user experience.
- Payment gateways need to remain accessible and functional to facilitate smooth and secure transactions.
- The project assumes cooperation from the campus administration in providing a server for deployment.
- Successful collaboration with local restaurants and riders is assumed for the platform to function effectively, ensuring timely order fulfillment and delivery.
- The system relies on the availability and reliability of third-party APIs.

3.6 Functional Requirements

The system provides mobile and web applications enabling users to register, update profiles, browse menus, and place orders. Vendors can manage stores

and menus, accept orders, and update order statuses. Additional modules support digital payments, delivery tracking, and a review forum with star ratings, filters, and search for enhanced user experience.

Functional Requirement No.	Functional Requirement Description
Mobile/Web Applications	
FR-1	New users can register profile
FR-2	Users can update profile
FR-3	Vendors can create an account to register store
FR-4	The vendor can update store details
E-Food Marketplace Module	
FR-5	Vendors can add/remove menus
FR-6	Vendor can update items
FR-7	Users can search for an item
FR-8	Users can add/remove items to and from cart
FR-9	Users can place an order
FR-10	User can complete payment transaction
FR-11	Vendors can accept orders
Review Forum Module	
FR-12	User can post review
FR-13	Users can give star ratings out of five
FR-14	Users can view listing of reviews with filter/sort/search options
Delivery Status tracking Module	
FR-15	User can track the order
FR-16	Vendors can update order status
FR-17	Order completion

Table 1 Functional Requirements

Requirement ID	**FR 1**		**Requirement Type**		Functional		**Use Case #**		0
Status	***New***	X	***Agreed-to***	-	***Baselined***	-	***Rejected***	-	
Parent Requirement #	None								
Description	The system must allow users to register with a valid email and password.								
Rationale	Each use case of the application will require the user to be registered. For the Review module, this will enable better services.								
Source	-				**Source Document**		-		
Acceptance/Fit Criteria	Ensure that the module collects and stores the data according to the type of user and the user credentials are secure.								
Dependencies	The user must provide their verified email and other information correctly. MongoDB (for storing user credentials and profiles), Node.js (backend), Password Encryption Module.								
Priority	***Essential***	X	***Conditional***	-	***Optional***	-			
Change History	None								

Table 2 User Registration

Requirement ID	**FR-2**		**Requirement Type**		Functional		**Use Case #**		1
Status	***New***	X	***Agreed-to***	-	***Baselined***	-	***Rejected***	-	
Parent Requirement #	FR 1								
Description	Users should be allowed to update their information and retrieve their account in case they forget their password.								
Rationale	Users should be able to update their personal info like contacts and email.								
Source	-				**Source Document**		-		
Acceptance/Fit Criteria	The information is updated while maintaining consistency in the database. The account retrieval mechanism should be secure and not spoof able.								
Dependencies	Dependent on the parent requirement								
Priority	***Essential***	X	***Conditional***	-	***Optional***	-			
Change History	None								

Table 3 User Information Updating

Requirement ID	**FR-3**		**Requirement Type**		Functional		**Use Case #**		2
Status	***New***	X	***Agreed-to***	**-**	***Baselined***	**-**	***Rejected***	**-**	
Parent Requirement #	None								
Description	Vendors can open online stores by providing details such as their phone number, store location, email, and password.								
Rationale	This case allows sellers to set up their e-stores, expanding the reach of their products. Store and seller details will be utilized for tracking purposes in case any issues arise.								
Source	**-**				**Source Document**		**-**		
Acceptance/Fit Criteria	The information is inputted and stored correctly.								
Dependencies	The user must provide their verified email and other information correctly. MongoDB (for storing user credentials and profiles), Node.js (backend), Password Encryption Module.								
Priority	***Essential***	X	***Conditional***	**-**	***Optional***	**-**			
Change History	None								

Table 4 Vendor Registration

Requirement ID	**FR-9**		**Requirement Type**		Functional			**Use Case #**	3
Status	***New***	X	***Agreed-to***	-	***Baselined***	-	***Rejected***	-	
Parent Requirement #	FR-1								
Description	The system must allow users to place an order with the option for delivery or pickup.								
Rationale	Core functionality for DineHub to facilitate food ordering.								
Source	-				**Source Document**	-			
Acceptance/Fit Criteria	Orders are placed successfully, and users receive confirmation and status tracking.								
Dependencies	MongoDB (for storing order details), Node.js (API)								
Priority	***Essential***	X	***Conditional***	-	***Optional***	-			
Change History	None								

Table 5 Order placement

Requirement ID	**FR-10**		**Requirement Type**		Functional		**Use Case #**		4
Status	***New***	X	***Agreed-to***	-	***Baselined***	-	***Rejected***	-	
Parent Requirement #	FR-9								
Description	The system must facilitate secure payment processing through multiple methods such as SadaPay, JazzCash, Cash.								
Rationale	Allows users to complete transactions smoothly and securely.								
Source	-				**Source Document**		-		
Acceptance/Fit Criteria	Users can complete payments successfully and receive confirmation notifications.								
Dependencies	Payment Gateways, Node.js (backend)								
Priority	***Essential***	X	***Conditional***	-	***Optional***	-			
Change History	None								

Table 6 Payment Process

Requirement ID	**FR-11**		**Requirement Type**		Functional		**Use Case #**		5
Status	***New***	X	***Agreed-to***	-	***Baselined***	-	***Rejected***	-	
Parent Requirement #	FR-9								
Description	Vendors can accept orders depending on whether the items are available currently.								
Rationale	Gives the vendor the ability to reject an order if it is not available.								
Source	-				**Source Document**		-		
Acceptance/Fit Criteria	Vendor can successfully accept/reject orders.								
Dependencies	MongoDB (for storing order information), Node.js (backend)								
Priority	***Essential***	X	***Conditional***	-	***Optional***	-			
Change History	None								

Table 7 Vendor Order Acceptance

3.7 Nonfunctional Requirements & Software

3.7.1 System Attributes

- Peak-time operations should be managed effectively by the system through concurrent user processing.
- The system should possess adjustable capabilities for easy growth when user numbers along with vendor numbers and transaction volume increases.
- The system must adopt standard security measures through HTTPS or SSL security
- Users need a system that presents itself naturally and makes the interface user-friendly.
- The system codebase requires clear documentation while implementing best coding standards for smooth updates and debugging processes.
- Both Android devices starting from version 8 and iOS devices from version 12 will be supported by the mobile app.
- Users must be able to access the web portal through Chrome Firefox Safari while the design remains fully operational for different browser screen dimensions

3.7.2 Performance Requirements

- The system should maintain dependability through fault-resistant features which continue platform operations through server outages or network breakdowns.

- The system should manage its resources efficiently to maintain low CPU utilization while using limited memory capacity and network bandwidth capacity while sustaining peak user experience.
- The system needs to link up with external payment processors through its payment gateway integration. Every segment of the system must exhibit identical information between its multiple modules.
- System updates including order status should automatically appear throughout the platform without delay.

3.7.3 Security Requirements

Creating an account in the system necessitates signing up. Each account has to go through an approval process carried out by the administrator after registration, before access to login is allowed. Once an account is approved, users can log in to the account with their credentials consisting of username and password. Passwords are encrypted to guarantee security of the user data.

The system employs JSON Web Tokens (JWT) to manage user authentication and authorization. When the user logs into the server, a token will be given as a JWT and sent to the client. This token is to then be securely saved within the client-side, such that it will be sent in every request in the future. Using such a setup, the server can validate or verify either the user or vendors' identity and how access to some of their resources is being controlled.

To accomplish complete end-to-end security, the whole web application needs to work over HTTPS, secured with an SSL certificate. All data that is being exchanged between the client and server will be encrypted and protected against interception or tampering. This will also help in establishing trust, as the verification of identities of servers will be done, and man-in-the-middle attacks will be prevented.

Chapter IV: Project Design/Architecture

This diagram illustrates a web app architecture where code is pushed into GitHub by developers, and users log into the app through SSL-encrypted connections on laptops and smartphones. A reverse proxy, Nginx, on port 3000 passes incoming traffic to Docker containers within a Giki Server, which consists of Nginx for load balancing, a Node.js backend on port 3001 for business logic, and MongoDB on port 27017 for database storage. The Node.js backend drives functionality such as a ChatBot, Stripe payment options, and data handling by communicating with MongoDB, providing a scalable, secure, and modular system through containerization.

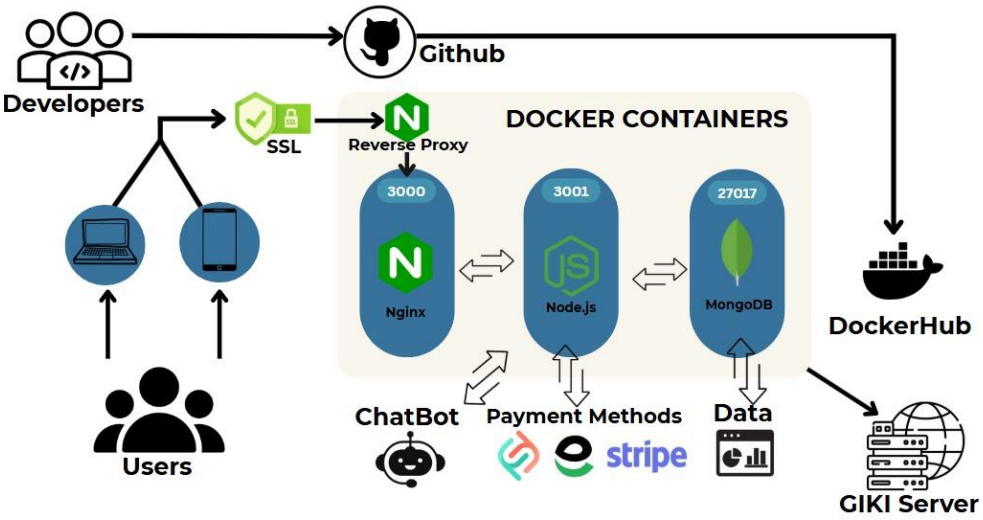


Figure 9 Architecture Diagram

4.1 Use Case View

4.1.1 User Use Case View

The following diagram illustrates a food ordering system where users sign up or log in with their name, password, and email to access the menu, search food items, add to cart, and submit orders with cash or online payment via an API, storing data in a database. Users can also view their order history.

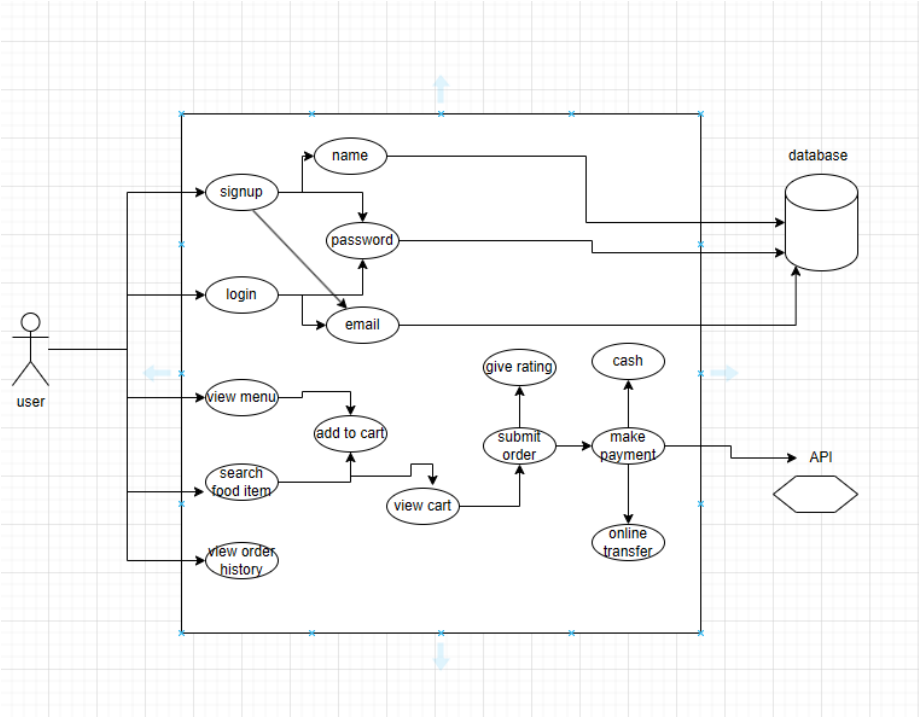


Figure 10 User Use Case View

4.1 Vendor Use Case View

At the same time, vendors create accounts, update menus, update items, verify order details, accept or decline orders, and pay through the API, and all of these are stored in the database.

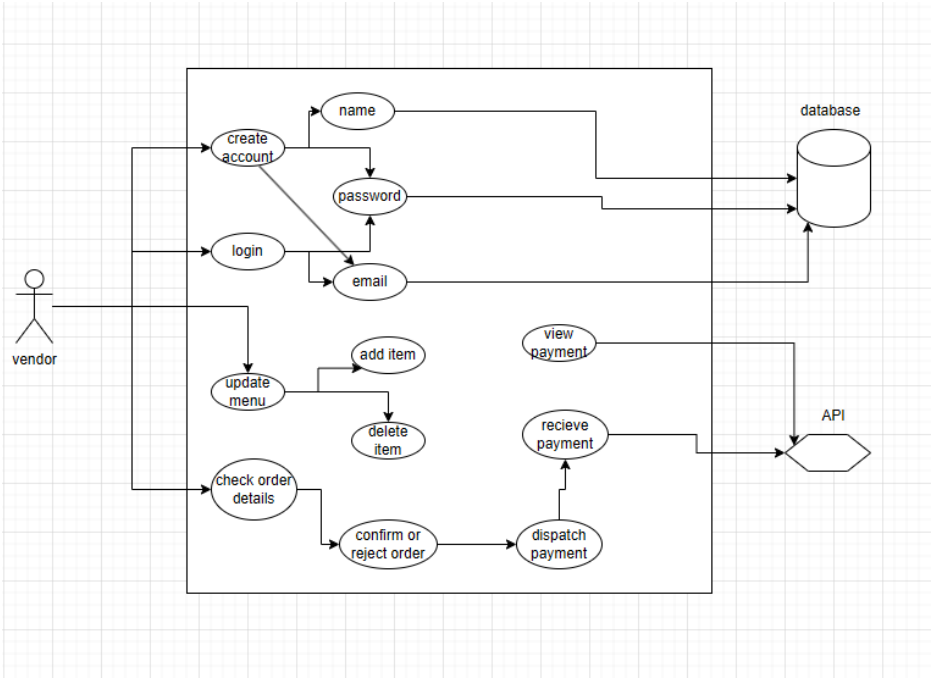


Figure 11 Vendor Use Case View

4.2 Logical View

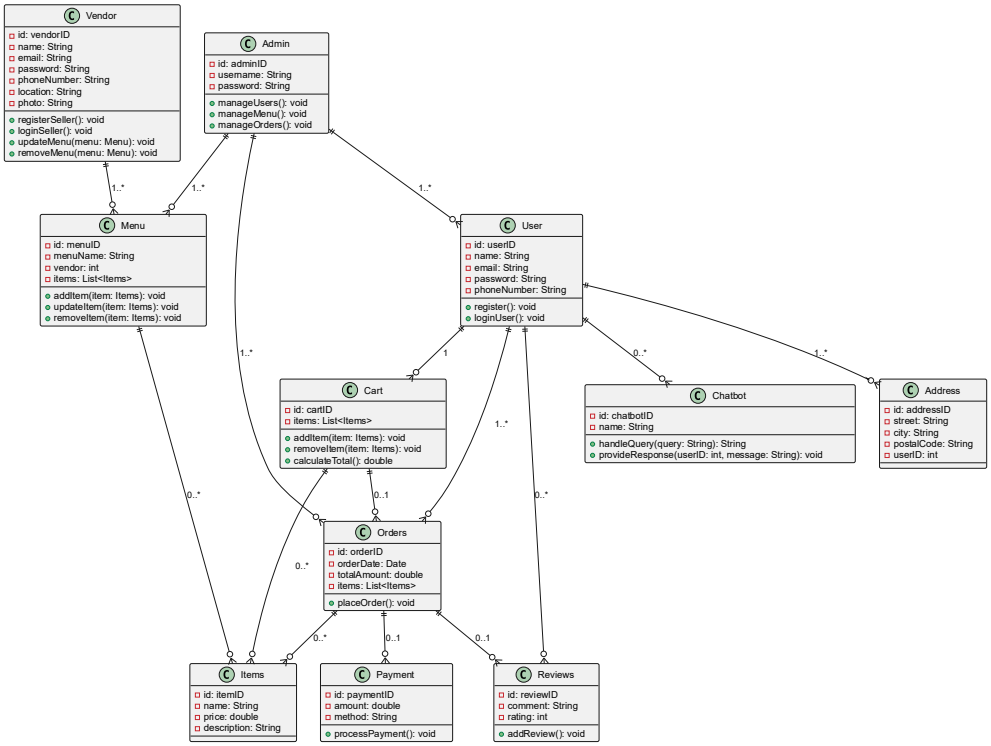


Figure 12 Class Diagram

4.3 Process View

4.3.1 User Sequence Diagram

During the process of ordering food, the User interacts through the User App. The user first goes through the menu browsing, adds items to the cart, checks the cart, logs in, and goes through the checkout process. After payment is made, the order is then submitted. After the delivery has been made, the user gets the chance to rate the experience. During each step, the User App interacts with the Order System to effect actions and retrieve real-time information

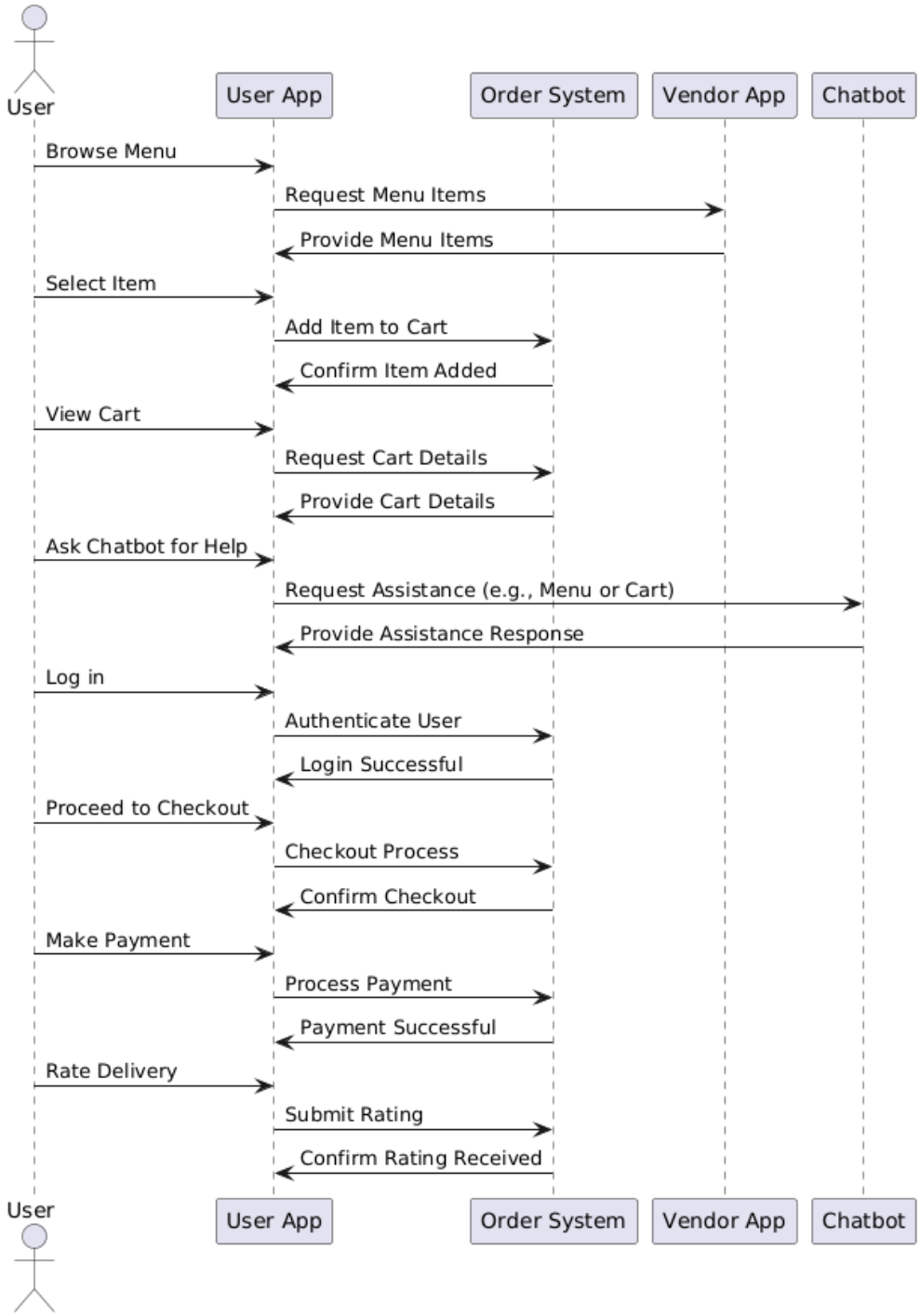


Figure 13 User Sequence Diagram

4.3.2 Vendor Sequence Diagram

On the seller side, the Vendor utilizes the Vendor App to manage orders. The seller logs in, views incoming orders, and verifies acceptance. Having prepared the dish, they mark the order ready and verify payment receipt. The Vendor App interacts in tandem with the Order System, which also updates the User App to inform the user on the progress in the process.

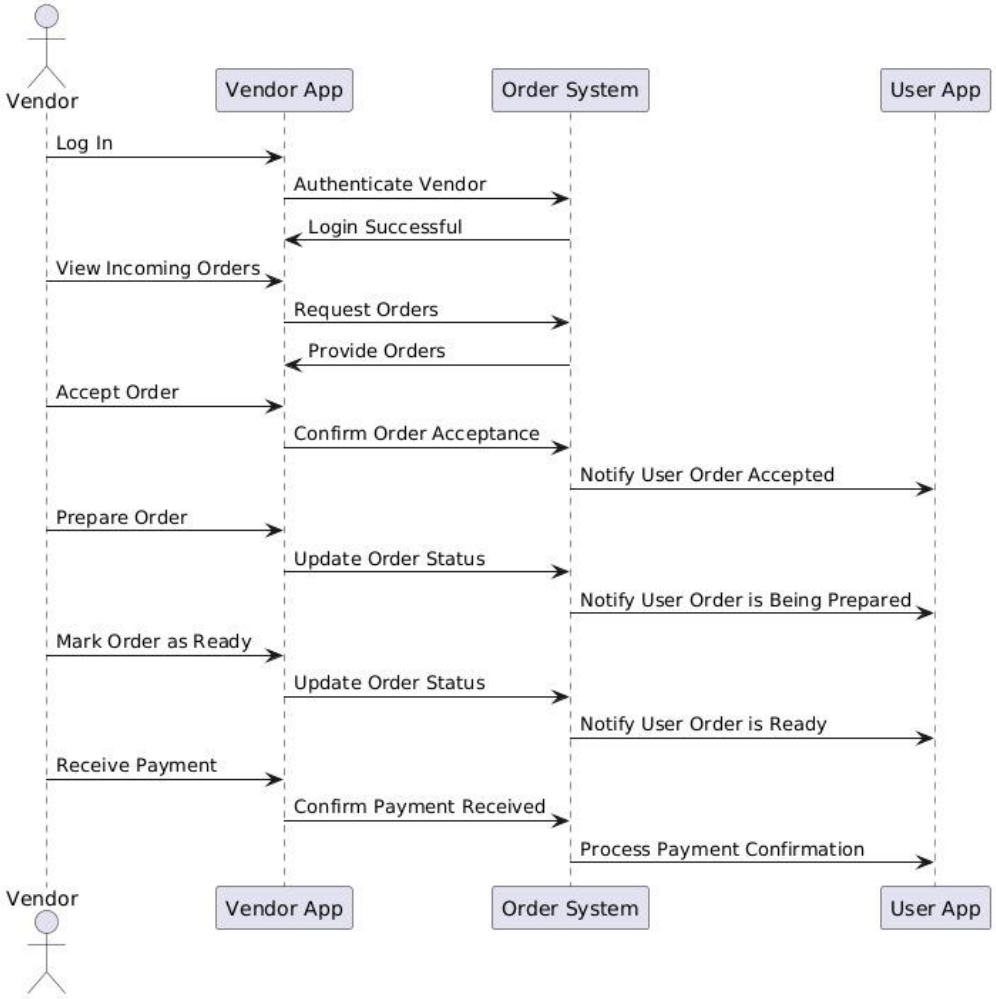


Figure 14 Vendor Sequence Diagram

4.4 Physical View

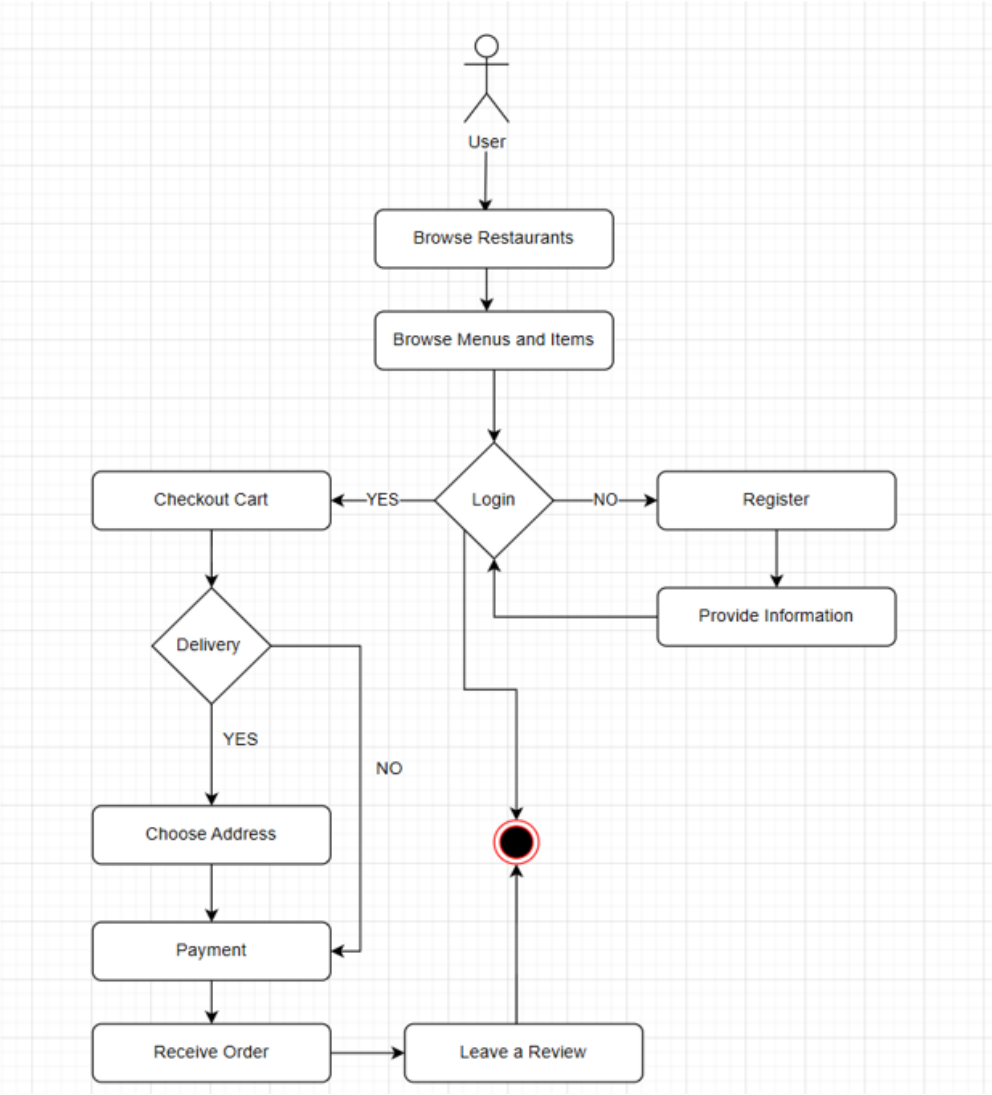


Figure 15 User Activity Diagram

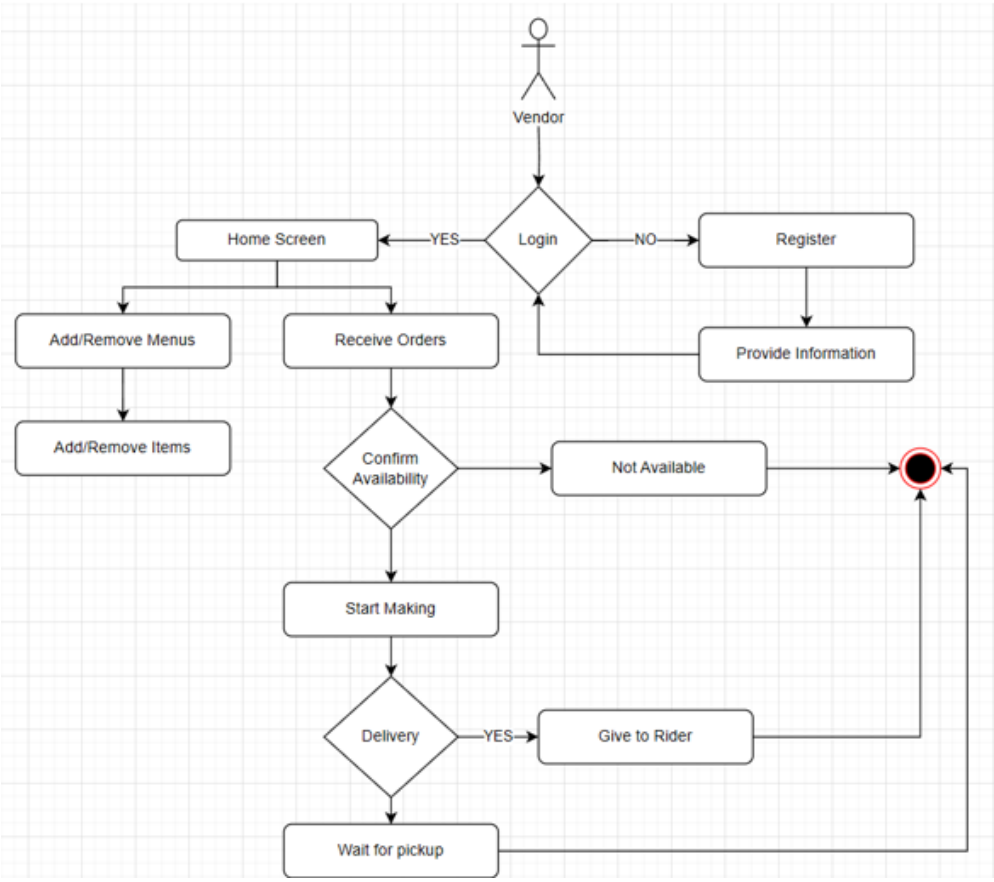


Figure 16 Vendor Activity Diagram

The flowchart illustrates the workflow for a vendor within the food ordering system, detailing the steps from login to order fulfillment. Starting at the Home Screen, the vendor can either log in or register if they are not already a user; registration requires providing necessary information, after which they are redirected to the login step. Upon successful login, vendors can manage their menus by adding or removing menus and items. Simultaneously, they can receive orders from customers, confirm the availability of the ordered items,

and proceed to prepare the order if items are available—if not, the process loops back to receiving orders. Once the order is prepared, the vendor decides whether to opt for delivery; if yes, the order is handed over to a rider, otherwise, the vendor waits for the customer to pick up the order, completing the cycle. This streamlined process ensures efficient order management and delivery coordination for vendors.

4.5 User Interface Design

4.5.1 User App

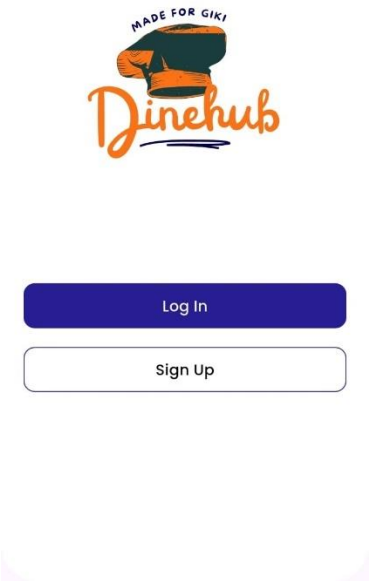


Figure 17 Flash Screen

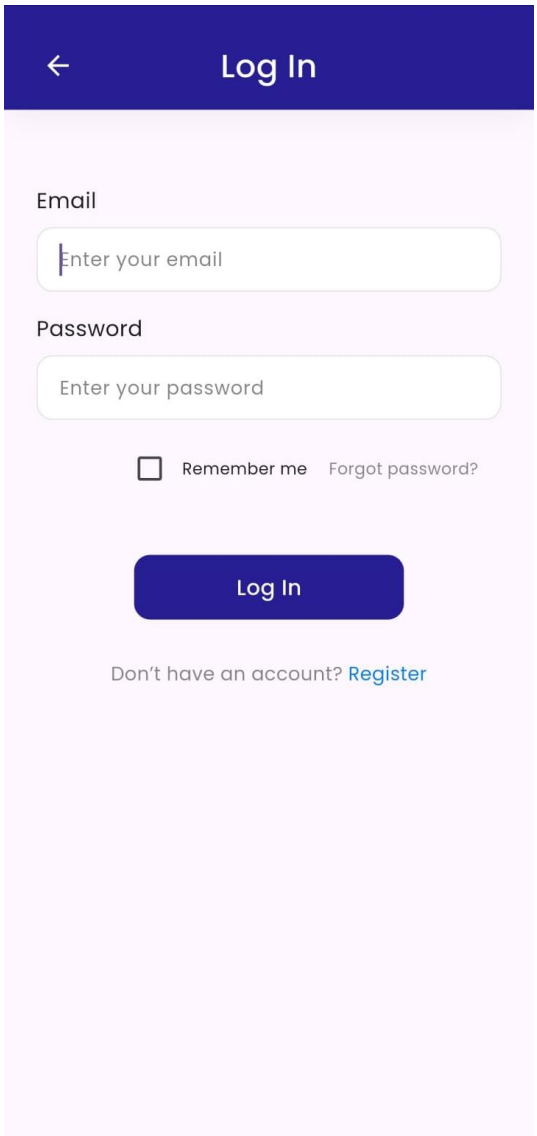


Figure 18 Login Page

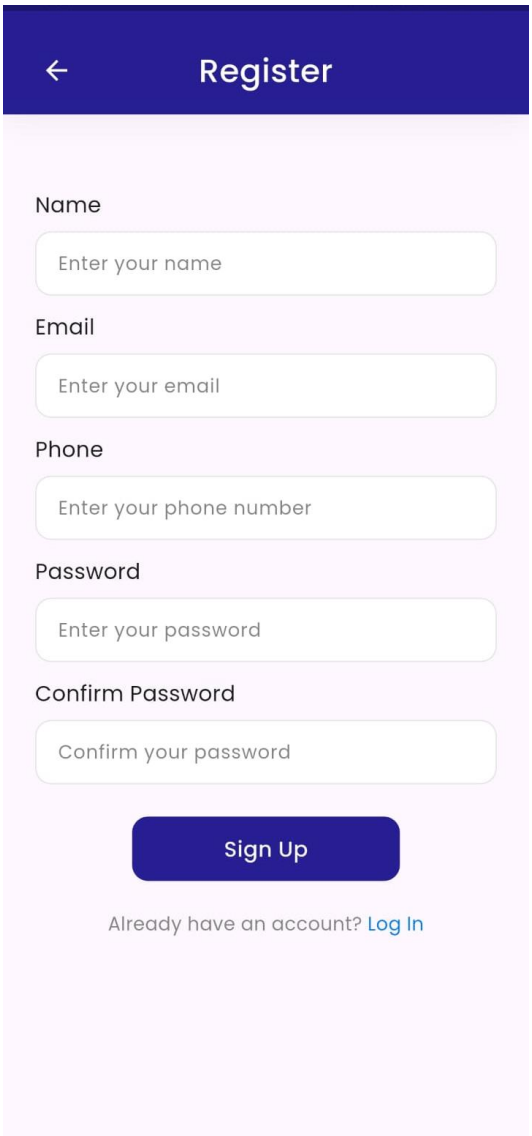


Figure 19 Register Page

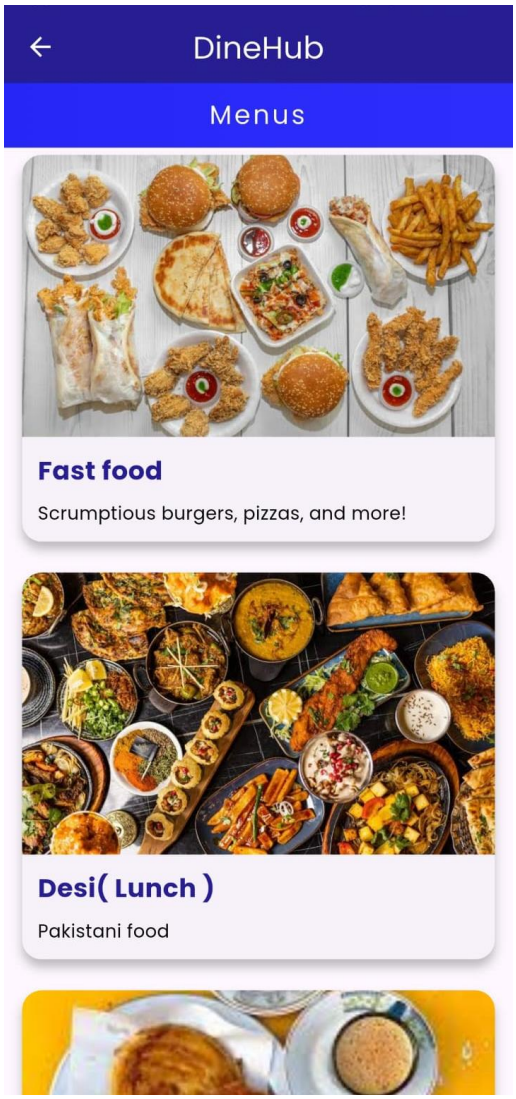


Figure 20 Menu Page

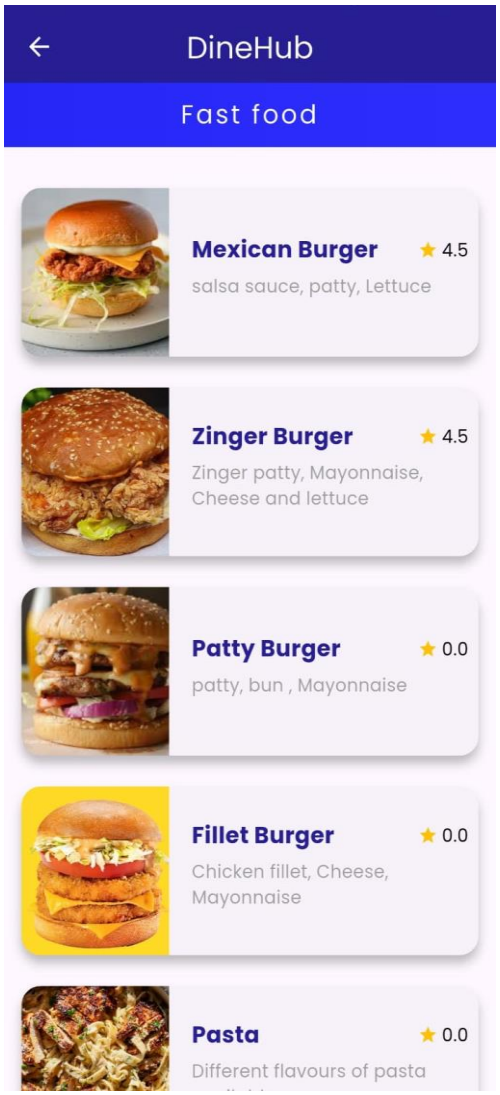


Figure 21 Items Page

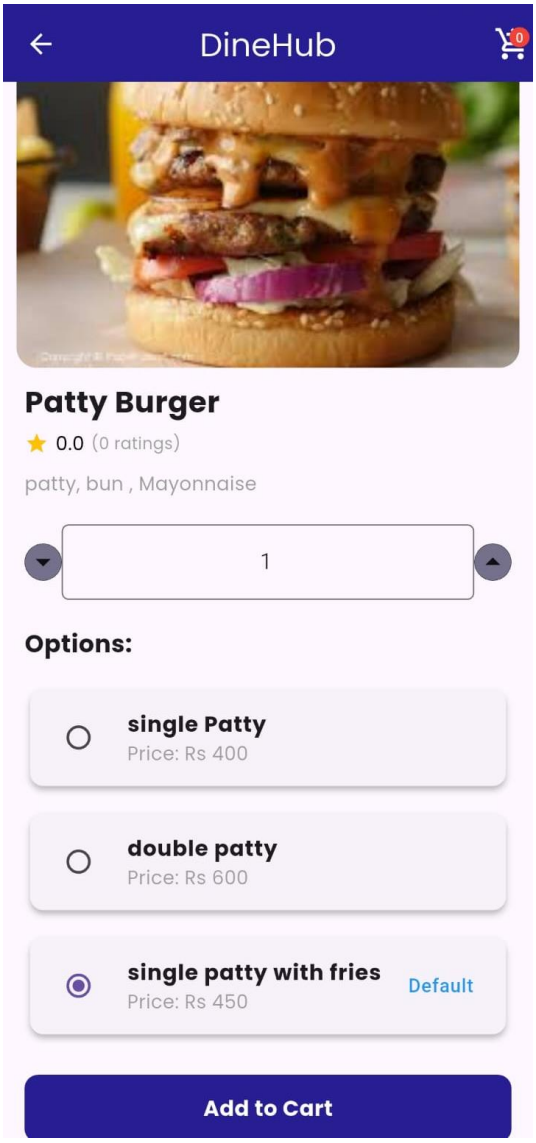


Figure 22 Item Details

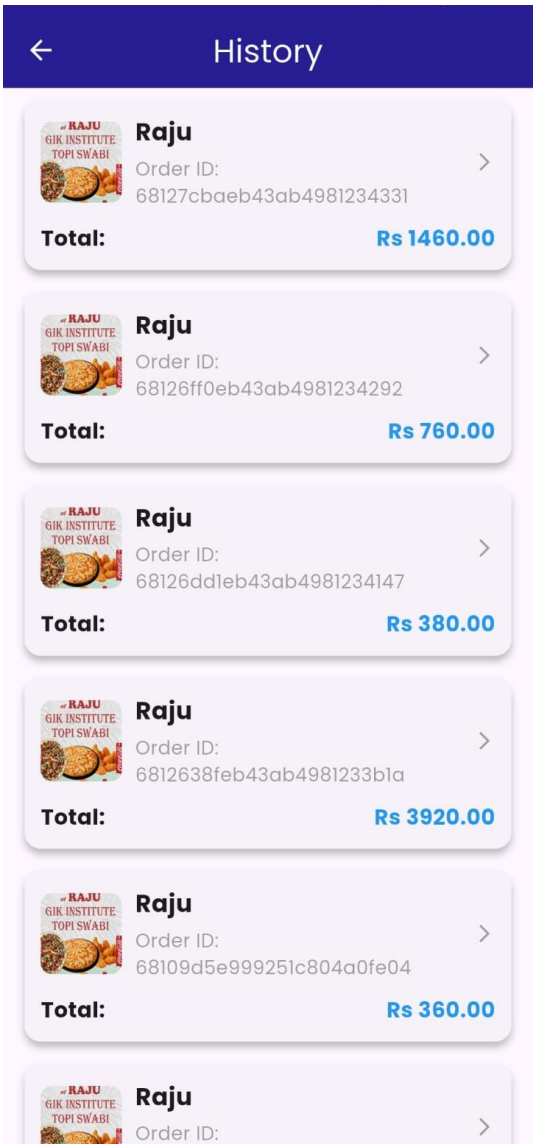


Figure 23 Order History

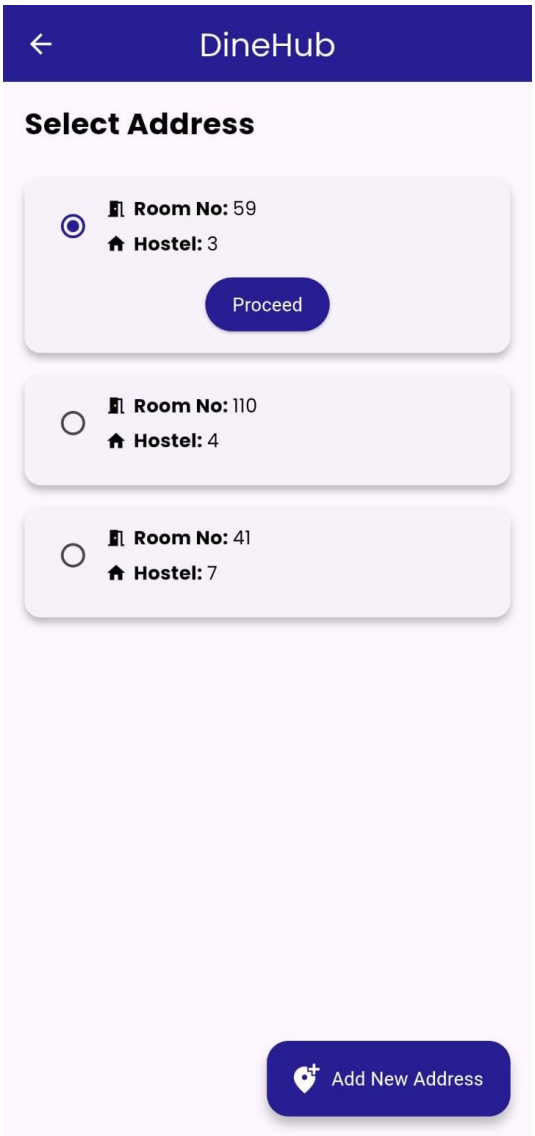


Figure 24 Address Page

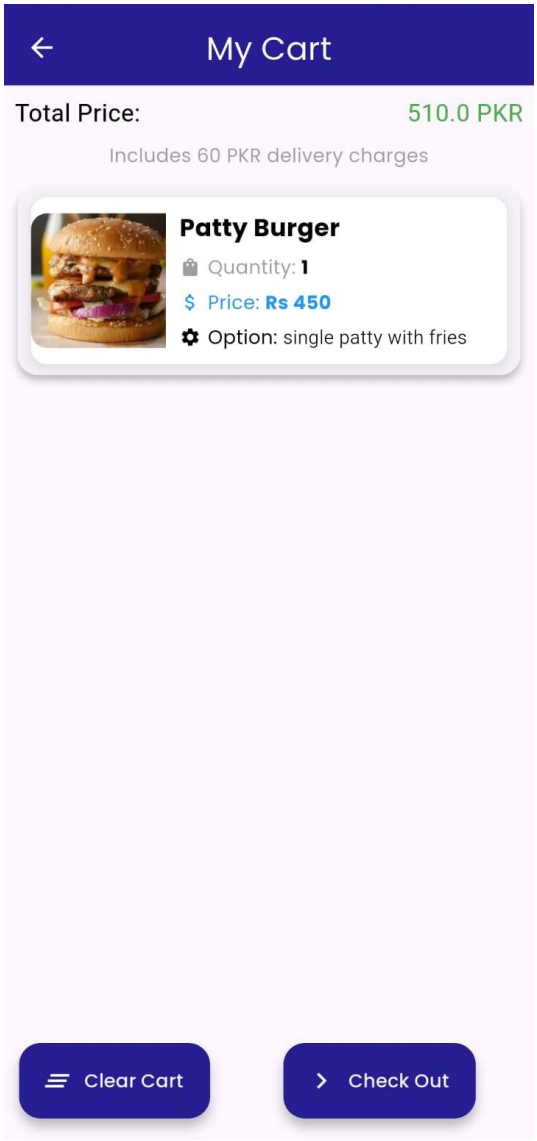


Figure 25 My Cart

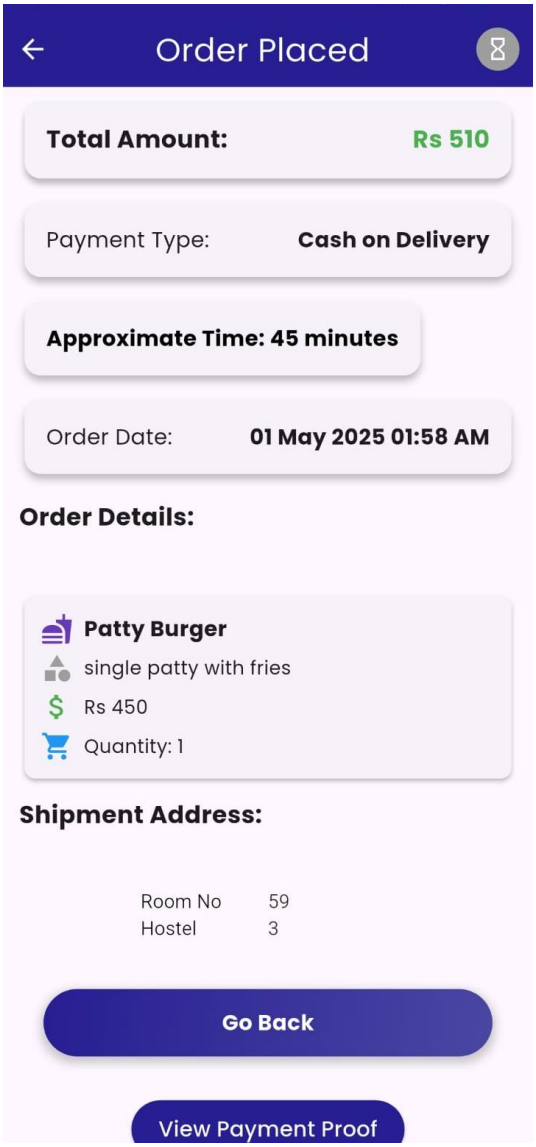


Figure 26 Order Place Screen

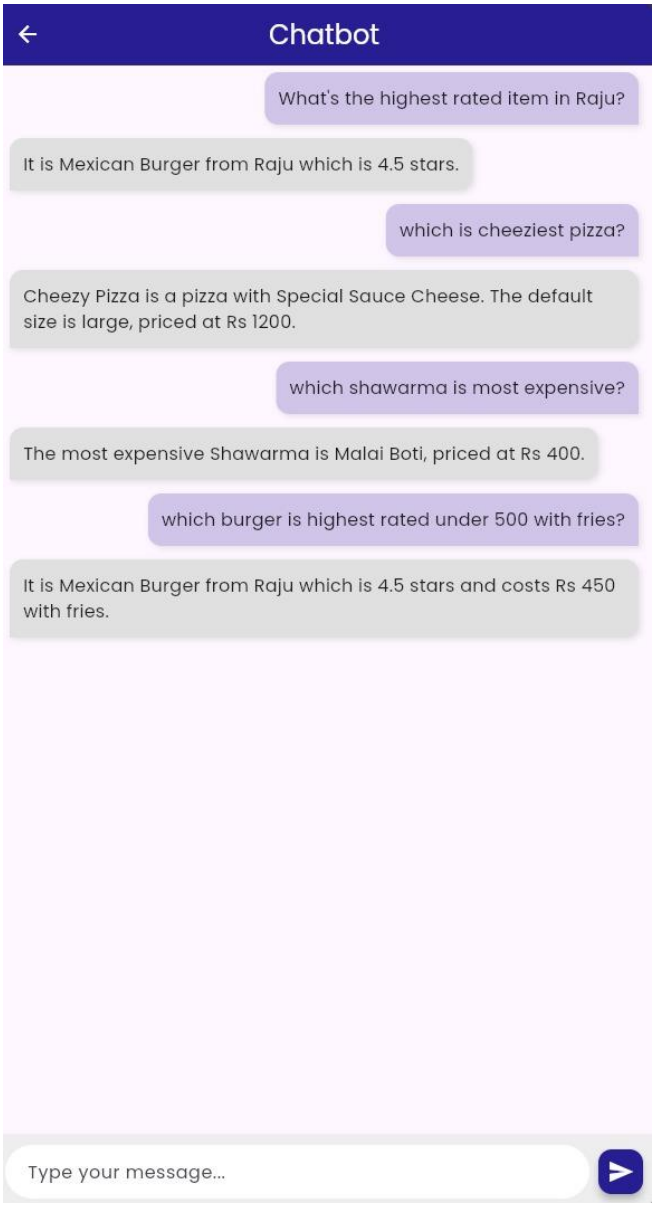


Figure 27 Chatbot Screen

4.5.2 Seller App

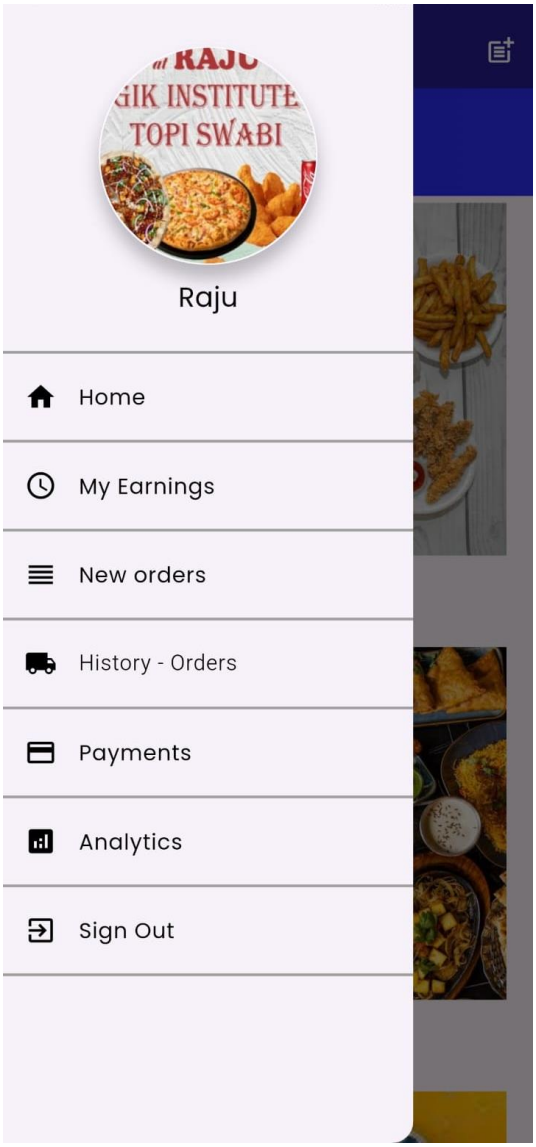


Figure 28 App Bar

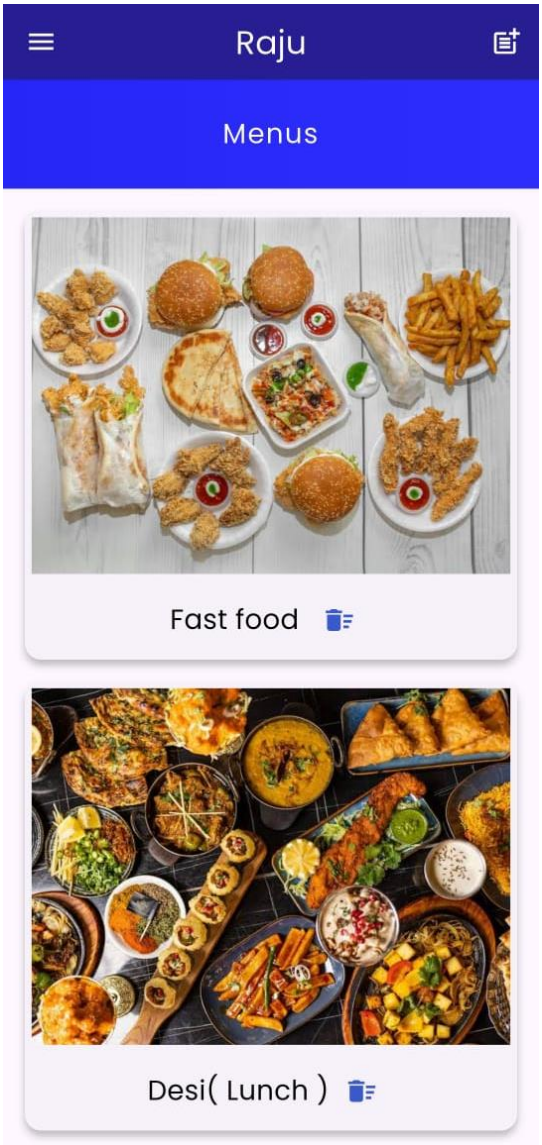


Figure 29 Menu Screen

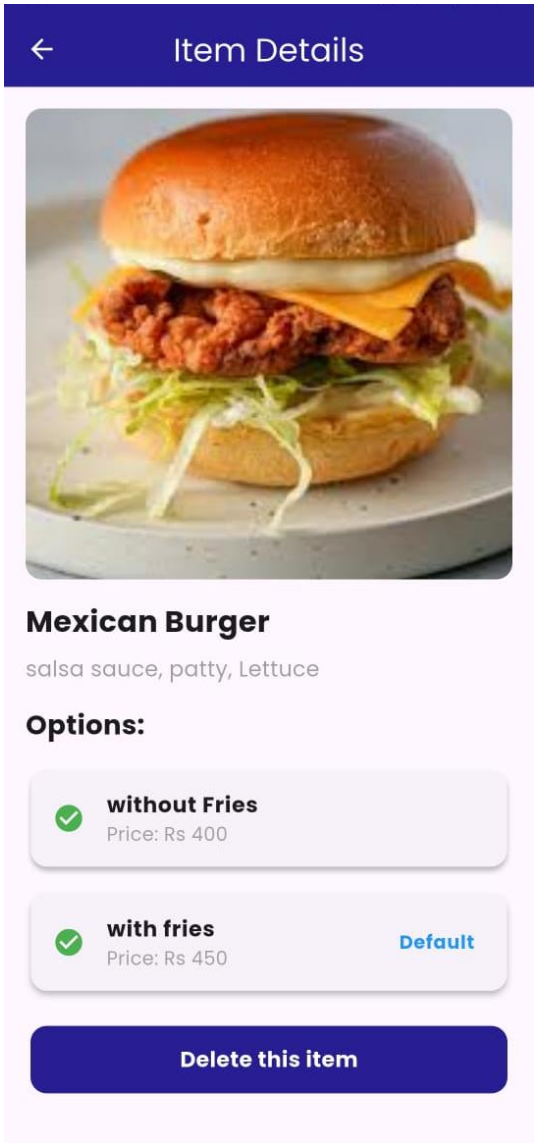


Figure 30 Item Details

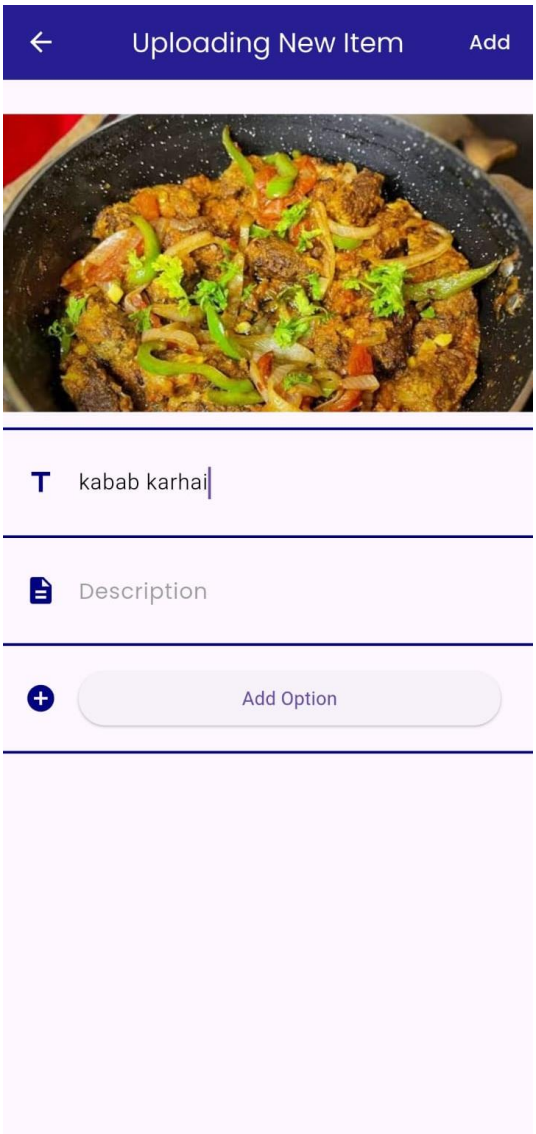


Figure 31 Uploading Items Screen

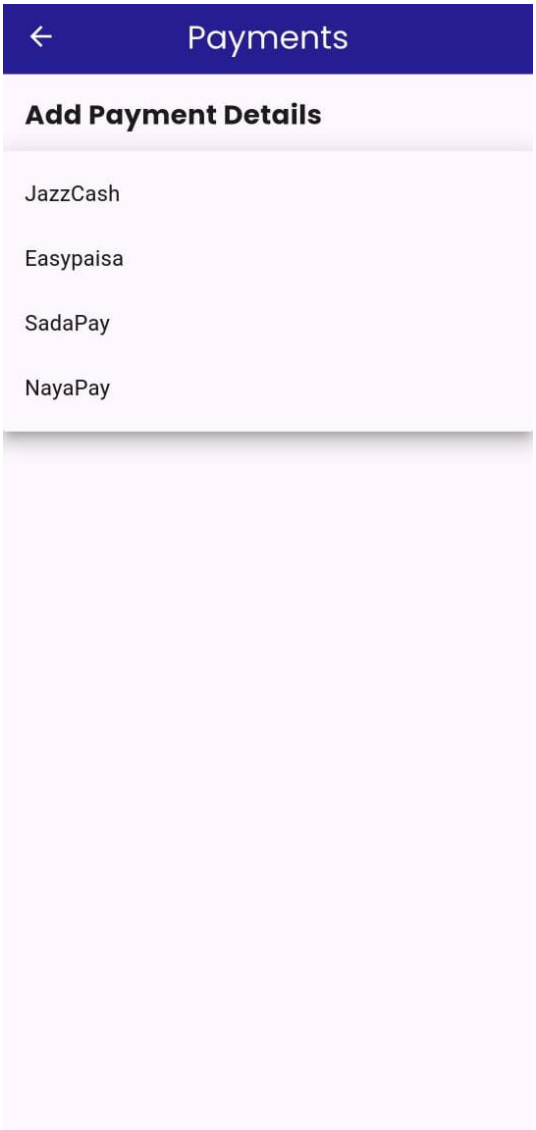


Figure 32 Payment Details Screen

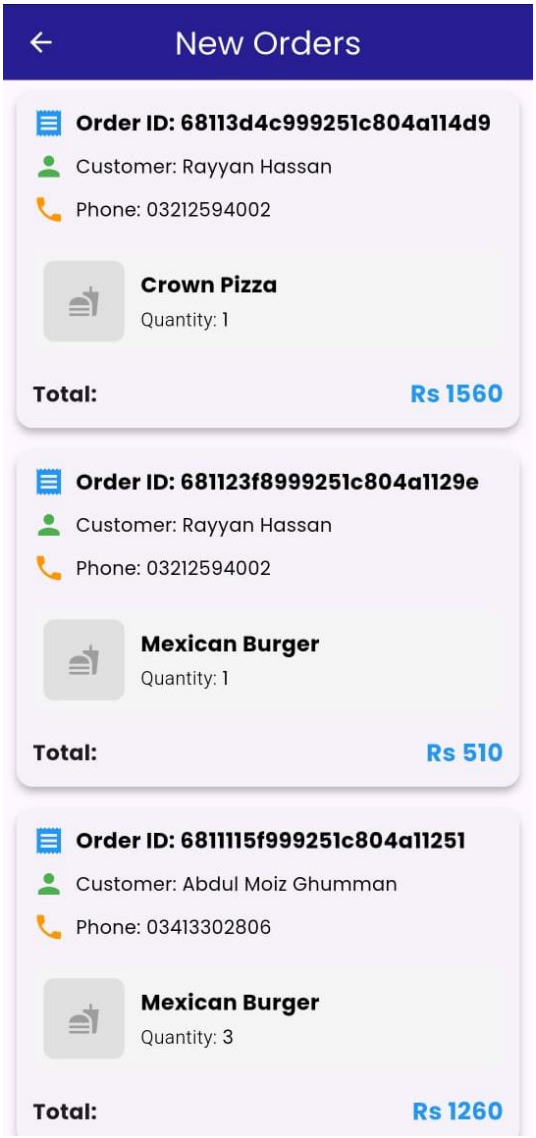


Figure 33 New Orders Screen

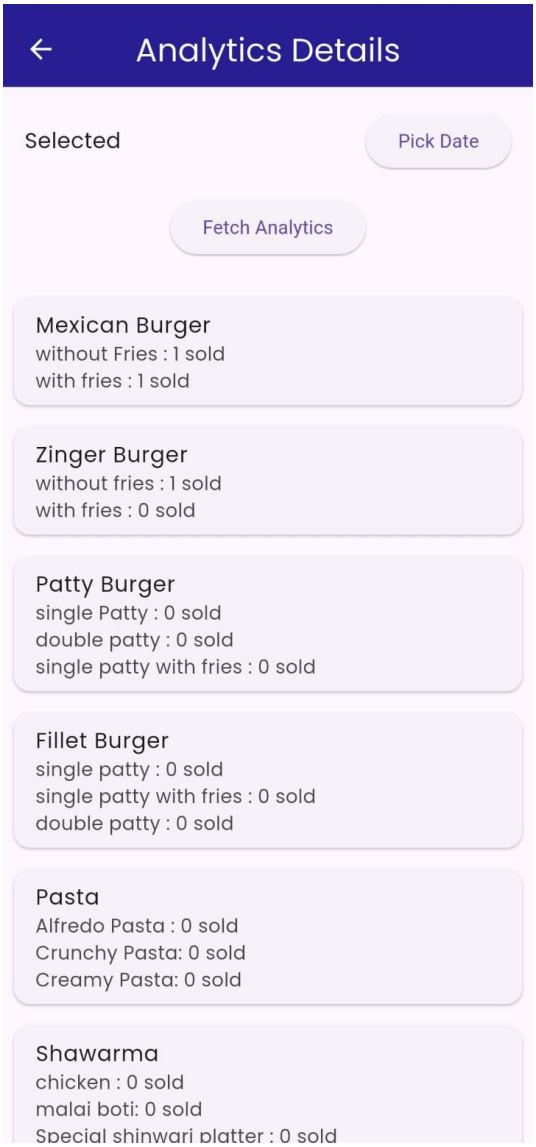


Figure 34 Analytics Screen

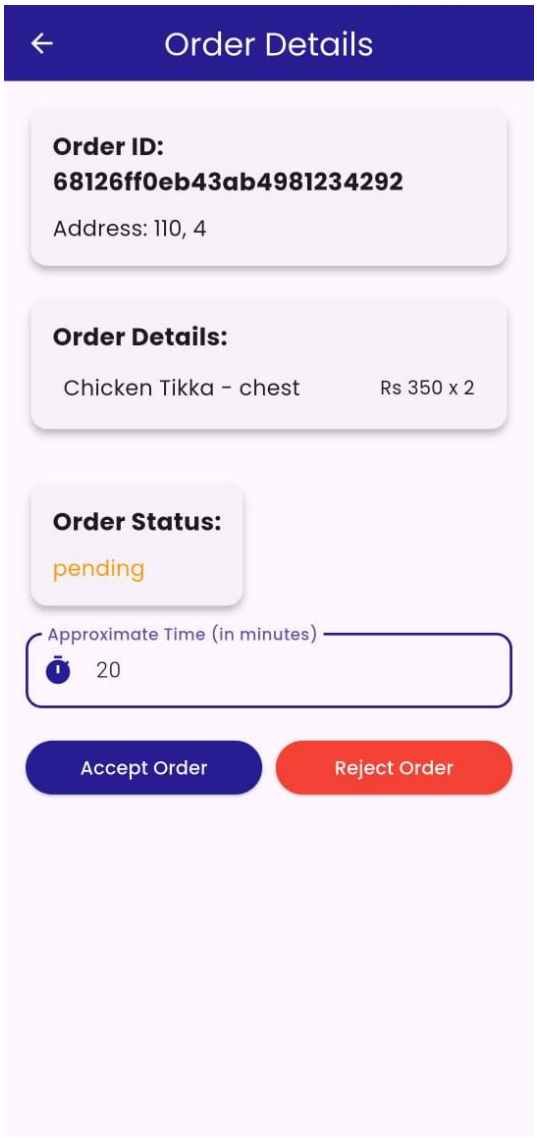


Figure 35 Order Details Screen

4.6 Context Diagram

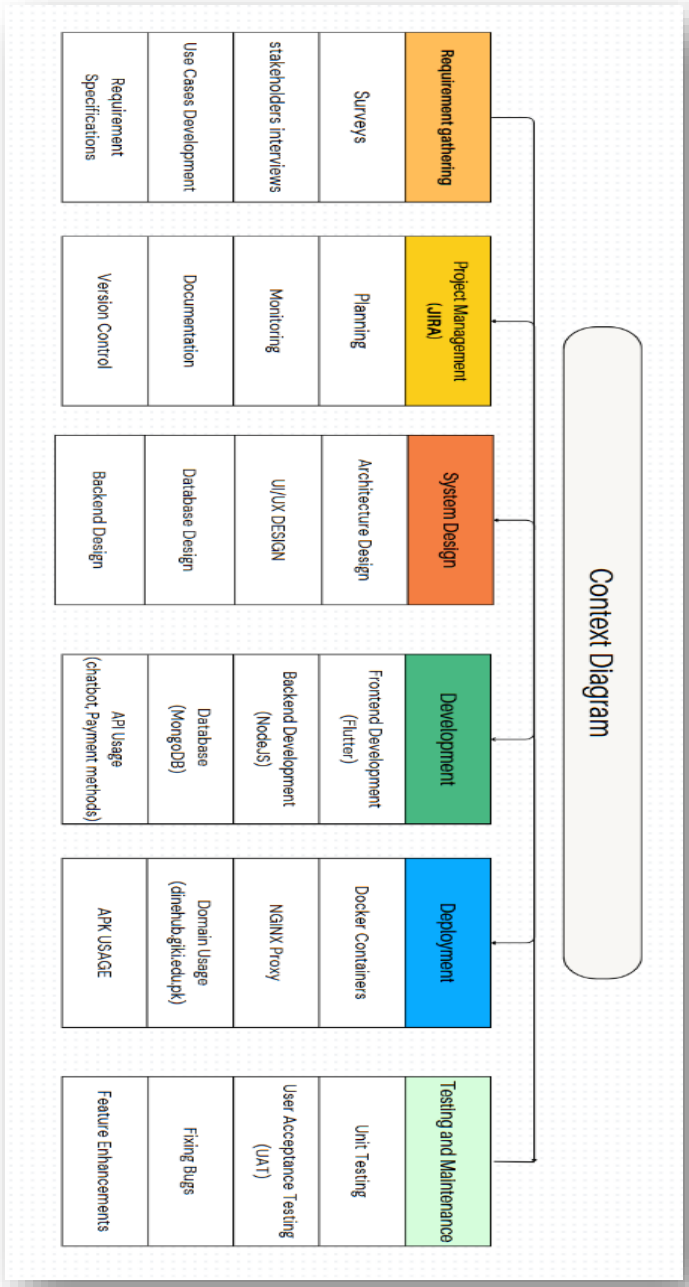


Figure 36 Context Diagram

Chapter V: Proposed Solution

5.1 Development Methodology

DineHub was created through the Agile Software Development Life Cycle (SDLC). We have structured weekly sprints with focused deliverables. Sprint planning, task management, and team member collaboration were done using Jira. An example screenshot is shown below:

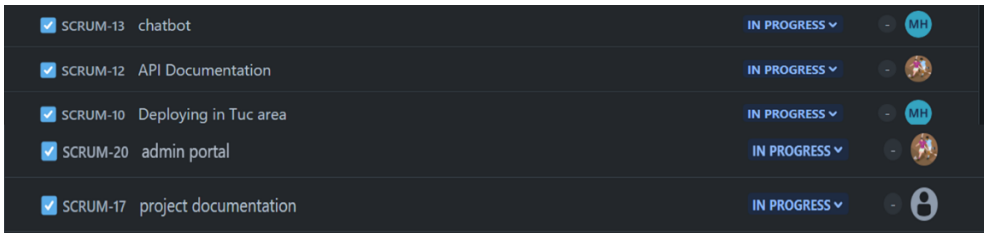


Figure 37 Jira Screenshot

The key phases were:

- Requirement gathering: Conducting surveys and interviews with potential users such as students, vendors, faculty, and riders to understand their pain points and desired features.
- UI/UX design in Figma: Designing intuitive, role-specific interfaces using **Figma**, incorporating feedback loops for early testing and adjustments.
- Integrating the backend and frontend: The system used RESTful APIs to communicate between the frontend (built in Flutter) and the backend (developed using Node.js and Express).
- Implementation of chatbot to enhance user experience: A Meta AI-powered chatbot was introduced to improve user support and offer intelligent recommendations.

- Deployment to a live server: hosted on a university server using Docker containers and NGINX.
- Testing and debugging: Performed at the end of each sprint to validate functionality, fix bugs, and ensure alignment with stakeholder needs.
- Evaluation and iteration based on user feedback.

5.2 System Implementation and Technologies

Layer	Technology Used
Frontend	Flutter
Backend	Node.js with Express.js
Database	MongoDB
Authentication	JWT (JSON Web Tokens) + bcrypt
Deployment	Docker, NGINX
API Testing and Documentation	Postman
Version Control	GitHub
Project Management	Jira
Security	SSL Certificates
Chatbot	Meta Llama 3 LLM

Table 8 Technology Stack

5.3 Containerized Deployment

The following container interactions are specified within a docker-compose.yml file that features a multi-container design that defines DineHub. We hosted the app on the GIKI server and employed Docker containers for scalable and modular deployment. Three individual containers are configured.

- Nginx: It acts as the entry point to the application. It is a reverse proxy that manages incoming HTTPS requests, routes requests to the desired container, and enforces SSL encryption using certificates.

- Node.js: Hosts the backend Express.js powering business logic. The server processes user requests (authentication, orders, menus, payments). It handles APIs for all user roles (users, vendors, and admins). connects with MongoDB for data persistence. Furthermore, it generates JWT tokens and validates user roles.

- Flutter-based frontend application: Built using **Flutter Web** and compiled to serve HTML/CSS/JS. Serves as the **user-facing interface** for all roles. Deployed in a container using a web server (typically **Nginx** or **http-server**) to serve the static web assets. Routes frontend API calls to the **backend Node.js** service via Nginx reverse proxy.

- MongoDB: The NoSQL database for storing users, orders, menus, and reviews. It ensures data reliability through **volume mounts** that persist data across container restarts. MongoDB supports real-time queries from the backend service.

```
docker@docker:~/dinehub/server_remote$ docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED         STATUS         PORTS                                                   NAMES
d94d19a79b04   nginx:latest       "/docker-entrypoint.…"   5 minutes ago   Up 5 minutes   80/tcp, 0.0.0.0:3000->3000/tcp, :::3000->3000/tcp       nginx_proxy
82868b74dffca  dinehub_snapshot   "docker-entrypoint.s…"   5 minutes ago   Up 5 minutes   0.0.0.0:3001->3000/tcp, :::3001->3000/tcp               backend
8007064e3d88   mongodb            "docker-entrypoint.s…"   5 minutes ago   Up 5 minutes   0.0.0.0:27017->27017/tcp, :::27017->27017/tcp           mongodb
docker@docker:~/dinehub/server_remote$ docker exec -it 828 sh
```

Figure 38 Containers

5.4 GitHub Repository and Collaboration

Figure 39 Contributors

All the source code is maintained in a GitHub repository, enabling seamless collaboration. The repository has different branches for frontend, backend, and a main branch to finalize the changes. This allowed for all group members to work on their own tasks.

DineHub: A food delivery platform for GIKI

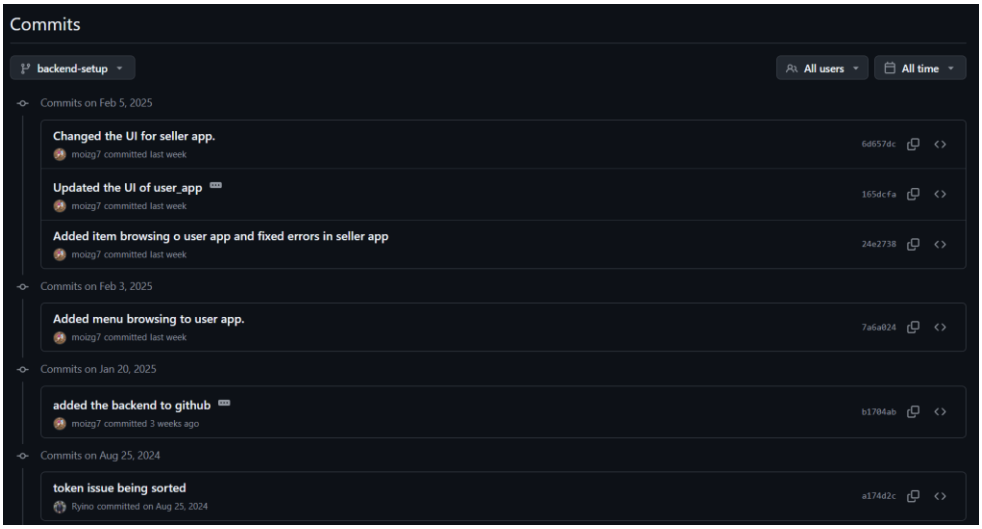


Figure 40 Commits Screenshot

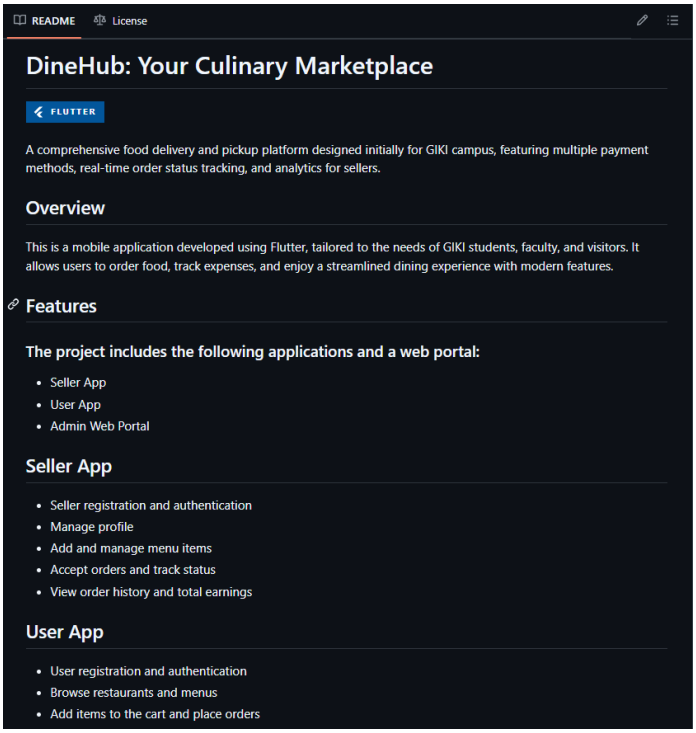


Figure 41 Readme File

5.5 API Documentation

The RESTful APIs, developed using ExpressJS, are meticulously documented and rigorously tested through Postman to ensure seamless functionality and accessibility for developers. The comprehensive documentation encompasses various key components, including authentication routes for secure user login and registration, user operations for managing profiles and interactions, order management for handling customer orders, vendor endpoints for menu and order processing, and admin-specific APIs for overseeing system operations. As illustrated in Figure 22, the documentation provides detailed examples of API calls, showcasing request formats, parameters, response structures, and potential error codes, thereby facilitating efficient integration and troubleshooting for developers working with the system. This structured approach ensures that all endpoints are well-defined, easily accessible, and thoroughly validated, enhancing the overall reliability and usability of the API ecosystem. [Postman Workspace]

Figure 42 Postman

5.6 Key Features and Security Measures

DineHub offers a range of features built with both functionality and security in mind:

- JWT-based authentication with role-based access control for users, vendors, and admins.
- Order status updates.
- Digital payments through SadaPay, JazzCash, and Easypaisa, along with support for cash-on-delivery.
- Push notifications and email alerts for order updates.
- SSL encryption (HTTPS) ensures secure data transmission.
- Password hashing with bcrypt to protect user credentials.
- Container isolation for backend, frontend, and database services
- Customized chatbot, suggesting the most popular orders. It also provides suggestions on what a user may want to order.

Chapter VI: Results and Discussions

6.1 System Implementation Outcomes

DineHub is built as a fully complete web application that runs in modular containers with Docker. It is tested with genuine users and under real-world campus situations.

Simplicity and responsiveness were given top priority during the development of the user interface. The Flutter-designed application provides web access as well as an APK file for Android users. Students and teachers can easily explore

menus, make orders, and follow deliveries because of the interface's good ability to adjust to different screen sizes and devices. The integration of a specially created chatbot assistant driven by Llama-3 improved the user experience, particularly for new users, by offering contextual help and suggestions.

The system has a secure role management and authentication system on the backend. Users, vendors, and administrators may all access the system via a single sign-on method thanks to JWT-based login features. Because each job is rigorously divided, no illegal acts can be carried out. To further protect user data, passwords are hashed using bcrypt, a widely recognized security technology. From the time an order is placed until it is delivered or picked up, the backend also makes the entire order lifecycle easier. While vendors can dynamically indicate order statuses through their specialized dashboards, consumers can check the development of their orders in real-time.

The combination of email alerts and push notifications was a very creative feature. These technologies provided real-time updates on order statuses, modifications, and promotions to both users and suppliers. In addition to increasing openness, these notifications sped up response times, especially during hectic order cycles.

Docker and NGINX were used to manage the actual deployment on a server that was given by GIKI. The application's frontend, backend, and database all operated in separate contexts thanks to the containerized deployment. This modularity greatly decreased inter-service conflicts and made debugging easier. Additionally, to safeguard all user-system connections, SSL encryption was implemented via HTTPS, exhibiting best practices in web application security.

6.2 User Testing

Pilot testing was carried out with genuine users in authentic campus settings to confirm the system's resilience. Students noted a discernible increase in customer satisfaction, accuracy, and ordering speed. The technique, according to vendors, significantly decreased order confusion and freed them up to concentrate more on food preparation rather than answering calls or negotiating payments. With the use of tools designed especially for governance and system control, administrators were able to easily monitor platform performance.

- Students: Reported a smoother, more reliable food ordering experience and appreciated the ability to browse menus and pre-order meals, reducing wait times.
- Vendors: Reported reduced confusion and improved order accuracy and found the dashboard helpful for tracking orders and managing items.

Overall, users found DineHub to be significantly more efficient than traditional phone call-based systems and appreciated the transparency and automation. Stress testing and load simulation confirmed that the system is capable of handling:

- 50+ concurrent users without significant delay.
- Efficient routing and processing via NGINX reverse proxy.
- Smooth data flow between services using Docker’s container networking.

6.3 User Feedback

We asked our customers for feedback on the application and how this makes their life easier and for the improvements needed and the feedback is listed below:

- Now that it's so simple, I just have to download the app, look at the menu, and then order without needing to make a phone call.
- This application has greatly simplified daily tasks at GIKI. There is no longer any necessity to chase vendors or seek help from friends for ordering.
- I no longer need to constantly ask repeatedly about availability or prices of products, as all the necessary information is at hand within the app.
- I love the chatbot function because it recommends foods to me based on my regular ordering patterns.
- "Paying through the app is that easy. I do not need to carry money around or worry about change."
- "Having everything I need in one spot menus, orders, payments, even reviews. It's made ordering food a heck of a lot less stressful."

6.3 Challenges Faced

While the project was a success, several challenges were encountered:

- API Integration: Payment gateway integration was time-consuming due to limited documentation.
- Real-time Deployment: We have made various semester projects so far, but never deployed a project; it was more challenging.

- Cross-platform Responsiveness: Ensuring consistent behaviour across multiple screen sizes and devices required significant UI adjustments.
- Deployment Environment: Debugging SSL and Docker configurations on the server took longer than expected, especially with reverse proxy routing.

Chapter VII: Conclusion and Future Work

7.1 Conclusion

The absence of a centralized, digital meal ordering system on the GIKI campus has been successfully addressed by DineHub's design and execution. DineHub improves convenience for students, instructors, vendors, and delivery personnel by eliminating phone calls, reducing order confusion, enabling digital payments, and providing real-time tracking.

Built with modern technologies like Node.js, MongoDB, Docker, and NGINX, and with Postman-documented and GitHub-governed APIs, the platform offers a safe and scalable environment for future growth.

DineHub is more than simply a project; it is an applied solution that provides real advantages to daily campus life by bridging the gap between software engineering theory and real-world implementation. This fulfils our objective from the beginning, which was to use an FYP as a chance to solve an issue rather than to complete it as a requirement.

7.2 Future Work

The introduction of a loyalty and rewards program to promote recurring business through a point-based system with discounts, coupons, or freebies is one of DineHub's upcoming improvements. With just minor architectural adjustments, the platform ought to search for cross-institutional expansion to other campuses. Lastly, to support data-driven decision-making, advanced admin analytics will be included, including AI-driven dashboards to help with insights on order trends, vendor performance, and payment patterns.

GLOSSARY

API **(Application Programming Interface):** A set of definitions and protocols that allows one piece of software to communicate with another.

Backend**:** The server-side part of the application responsible for business logic and database operations.

Docker**:** A containerization tool used to deploy and run applications in isolated environments.

JWT **(JSON Web Token):** A secure way of transmitting user authentication data as a digitally signed token.

MongoDB**:** A NoSQL document-based database used for storing flexible and scalable data structures.

NGINX**:** A high-performance web server used for load balancing and serving as a reverse proxy.

Node.js**:** A JavaScript runtime used for developing scalable and fast backend applications.

Postman**:** A platform used for developing, testing, and documenting APIs.

SSL **(Secure Sockets Layer):** A protocol used to encrypt data transferred between users and servers, ensuring secure communication.

REFERENCES

[1] Docker Documentation, *Docker Docs*, 2024. [Online]. Available: https://docs.docker.com

[2] GitHub, *GitHub—Version Control Platform*, 2024. [Online]. Available: https://github.com

[3] MongoDB, *MongoDB Documentation*, 2024. [Online]. Available: https://www.mongodb.com/docs

[4] Node.js, *Node.js Documentation*, 2024. [Online]. Available: https://nodejs.org/en/docs

[5] Postman, *Postman API Platform*, 2024. [Online]. Available: https://www.postman.com

[6] NGINX, *NGINX Documentation*, 2024. [Online]. Available: https://docs.nginx.com

[7] Express.js, *Express - Node.js web application framework*, 2024. [Online]. Available: https://expressjs.com

[8] Flutter, *Flutter Documentation*, 2024. [Online]. Available: https://docs.flutter.dev

[9] Dart, *Dart Programming Language*, 2024. [Online]. Available: https://dart.dev

[10] Stripe, *Stripe API Documentation*, 2024. [Online]. Available: https://stripe.com/docs/api

[11] Firebase, *Firebase Documentation*, 2024. [Online]. Available: https://firebase.google.com/docs

[12] JWT.io, *JSON Web Tokens - Introduction*, 2024. [Online]. Available: https://jwt.io/introduction

[13] DigitalOcean, *App Platform Documentation*, 2024. [Online]. Available: https://docs.digitalocean.com

[14] Heroku, *Heroku Dev Center*, 2024. [Online]. Available: https://devcenter.heroku.com

[15] OWASP, *OWASP Top Ten Web Application Security Risks*, 2024. [Online]. Available: https://owasp.org/www-project-top-ten/

[16] Google Developers, *Web Performance Optimization*, 2024. [Online]. Available: https://developers.google.com/web/fundamentals/performance

[17] OpenAI, *OpenAI API Documentation*, 2024. [Online]. Available: https://platform.openai.com/docs

[18] Google Fonts, *Fonts for the Web*, 2024. [Online]. Available: https://fonts.google.com

[19] Material Design, *Material Design Guidelines*, 2024. [Online]. Available: https://m3.material.io

APPENDIX

A.1 Jira Project Board

(Fig. 37)—Snapshot of task assignment during Agile development.

A.2 Docker Containers

(Fig. 38)—Structure showing Dockerized services for MongoDB, Node.js, and NGINX.

A.3 GitHub Repository

(Fig. 20, Fig. 41)—Contributors' overview, commit history.

A.4 API Testing

(Fig. 42)—Postman API requests samples and organized collections for documentation and testing.